

Đồ Thị Không Trọng Số

BỒI DƯỠNG KIẾN THỨC NỀN CP

Đỗ Phan Thuận



Email/Facebook: thuandp.sinhvien@gmail.com

Khoa Công nghệ thông tin
Trường Đại học Tôn Đức Thắng.

Ngày 25 tháng 10 năm 2025

- 1 Cơ bản về đồ thị
- 2 Tìm kiếm theo chiều sâu và ứng dụng - DFS
- 3 Tìm kiếm theo chiều rộng và ứng dụng - BFS

1 Cơ bản về đồ thị

- Khái niệm
- Biểu diễn đồ thị
- Một số tính chất cơ bản

2 Tìm kiếm theo chiều sâu và ứng dụng - DFS

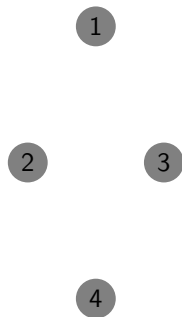
3 Tìm kiếm theo chiều rộng và ứng dụng - BFS

Đồ thị là gì?

Đồ thị là gì?

- Đỉnh

- ▶ Giao của các con đường
- ▶ Máy tính
- ▶ Các sà trong nhà
- ▶ Sân bay
- ▶ Các đối tượng



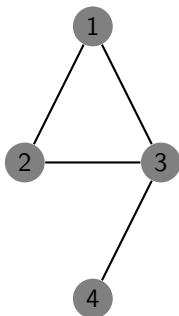
Đồ thị là gì?

- **Đỉnh**

- ▶ Giao của các con đường
- ▶ Máy tính
- ▶ Các sàn trong nhà
- ▶ Sân bay
- ▶ Các đối tượng

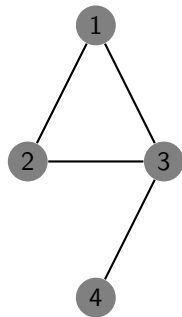
- **Cạnh**

- ▶ Con đường
- ▶ Dây mạng
- ▶ Thang bộ và thang máy
- ▶ Đường bay trực tiếp
- ▶ Mỗi quan hệ giữa các đối tượng



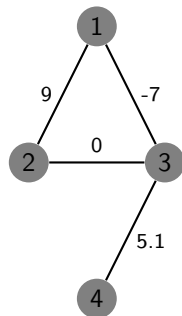
Các loại cạnh

- Không có trọng số



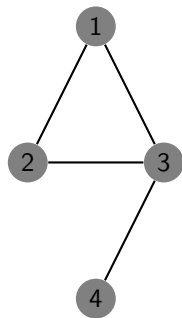
Các loại cạnh

- Không có trọng số hoặc Có trọng số



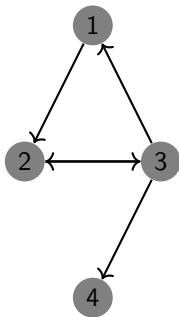
Các loại cạnh

- Không có trọng số hoặc Có trọng số
- Vô hướng



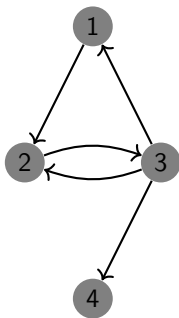
Các loại cạnh

- Không có trọng số hoặc Có trọng số
- Vô hướng hoặc **Có hướng**

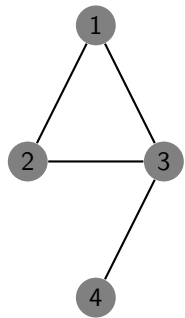


Các loại cạnh

- Không có trọng số hoặc Có trọng số
- Vô hướng hoặc Có hướng
- Trong trường hợp đồ thị có hướng, cạnh có hướng thường được gọi là cung chỉ tính định hướng của cung giữa hai đỉnh đầu nút

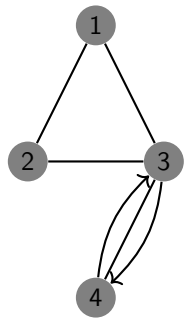


Đa đồ thị



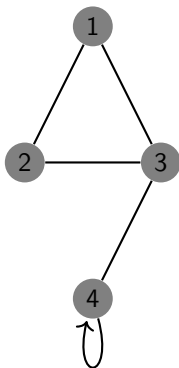
Đa đồ thị

- Cạnh lặp



Đa đồ thị

- Cạnh lặp
- Khuyến



1 Cơ bản về đồ thị

- Khái niệm
- Biểu diễn đồ thị
- Một số tính chất cơ bản

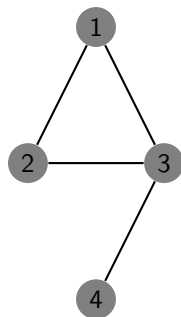
2 Tìm kiếm theo chiều sâu và ứng dụng - DFS

3 Tìm kiếm theo chiều rộng và ứng dụng - BFS

Danh sách kề

1: 2, 3
2: 1, 3
3: 1, 2, 4
4: 3

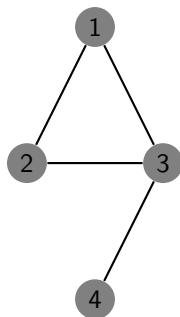
```
vector<int> Adj[5];  
Adj[1].push_back(2);  
Adj[1].push_back(3);  
Adj[2].push_back(1);  
Adj[2].push_back(3);  
Adj[3].push_back(1);  
Adj[3].push_back(2);  
Adj[3].push_back(4);  
Adj[4].push_back(3);
```



Mã trận kề

```
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
```

```
bool Adj[5][5];
Adj[1][2] = true;
Adj[1][3] = true;
Adj[2][1] = true;
Adj[2][3] = true;
Adj[3][1] = true;
Adj[3][2] = true;
Adj[3][4] = true;
Adj[4][3] = true;
```



Danh sách cạnh

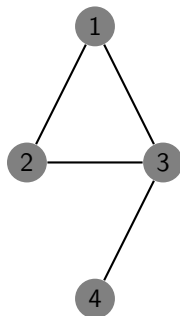
(1,2)

(1,3)

(2,3)

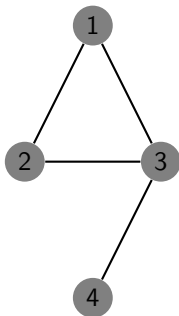
(3,4)

```
vector<pair<int, int> > Edges;  
Edges.push_back(make_pair(1,2));  
Edges.push_back(make_pair(1,3));  
Edges.push_back(make_pair(2,3));  
Edges.push_back(make_pair(3,4));
```



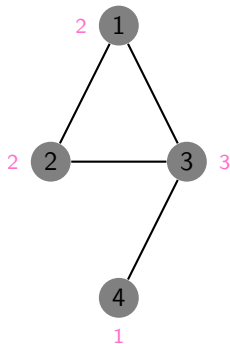
Một số tính chất trên đỉnh (đồ thị vô hướng)

- **Bậc của một đỉnh v , ký hiệu $\text{Deg}(v)$:**
 - ▶ Số lượng cạnh kề với v
 - ▶ Số lượng đỉnh kề với v



Một số tính chất trên đỉnh (đồ thị vô hướng)

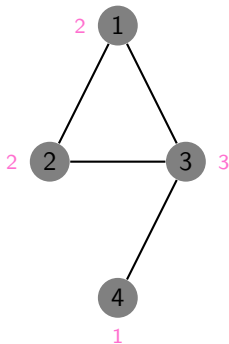
- **Bậc của một đỉnh v , ký hiệu $\text{Deg}(v)$:**
 - ▶ Số lượng cạnh kề với v
 - ▶ Số lượng đỉnh kề với v



Một số tính chất trên đỉnh (đồ thị vô hướng)

- Bậc của một đỉnh v , ký hiệu $\text{Deg}(v)$:
 - ▶ Số lượng cạnh kề với v
 - ▶ Số lượng đỉnh kề với v
- Bổ đề về những cái bắt tay (Handshaking theorem)

$$\sum_{v \in V} \text{Deg}(v) = 2|E|$$

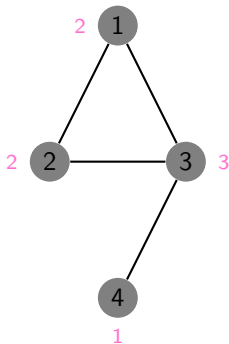


Một số tính chất trên đỉnh (đồ thị vô hướng)

- Bậc của một đỉnh v , ký hiệu $\text{Deg}(v)$:
 - ▶ Số lượng cạnh kề với v
 - ▶ Số lượng đỉnh kề với v
- Bổ đề về những cái bắt tay (Handshaking theorem)

$$\sum_{v \in V} \text{Deg}(v) = 2|E|$$

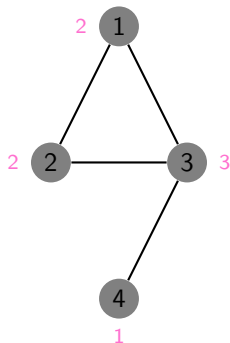
$$2 + 2 + 3 + 1 = 2 \times 4$$



Một số tính chất trên đỉnh (đồ thị vô hướng)

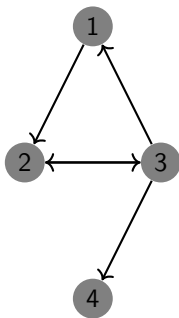
1: 2, 3
2: 1, 3
3: 1, 2, 4
4: 3

`Adj[1].size()` // 2
`Adj[2].size()` // 2
`Adj[3].size()` // 3
`Adj[4].size()` // 1



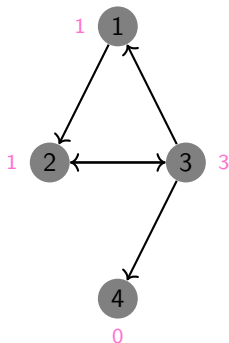
Một số tính chất trên đỉnh (đồ thị có hướng)

- Bán bậc ra của một đỉnh
 - ▶ Số lượng cạnh đi ra khỏi đỉnh



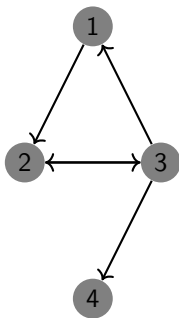
Một số tính chất trên đỉnh (đồ thị có hướng)

- Bán bậc ra của một đỉnh
 - ▶ Số lượng cạnh đi ra khỏi đỉnh



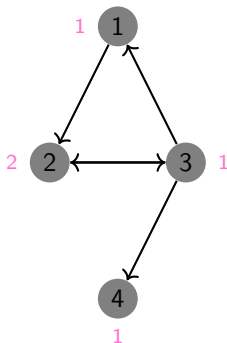
Một số tính chất trên đỉnh (đồ thị có hướng)

- Bán bậc ra của một đỉnh
 - ▶ Số lượng cạnh đi ra khỏi đỉnh
- Bán bậc vào của một đỉnh
 - ▶ Số lượng cạnh đi vào đỉnh



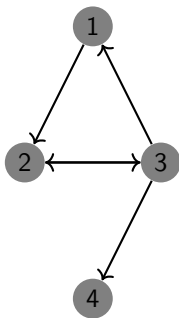
Một số tính chất trên đỉnh (đồ thị có hướng)

- Bán bậc ra của một đỉnh
 - ▶ Số lượng cạnh đi ra khỏi đỉnh
- Bán bậc vào của một đỉnh
 - ▶ Số lượng cạnh đi vào đỉnh



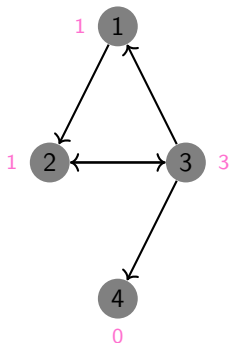
Một số tính chất trên đỉnh (đồ thị có hướng)

- Bán bậc ra của một đỉnh
 - ▶ Số lượng cạnh đi ra khỏi đỉnh
- Bán bậc vào của một đỉnh
 - ▶ Số lượng cạnh đi vào đỉnh



Một số tính chất trên đỉnh (đồ thị có hướng)

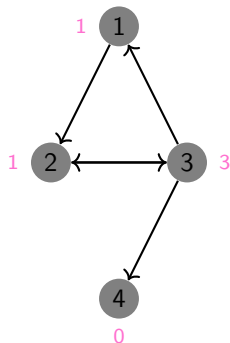
- Bán bậc ra của một đỉnh
 - ▶ Số lượng cạnh đi ra khỏi đỉnh
- Bán bậc vào của một đỉnh
 - ▶ Số lượng cạnh đi vào đỉnh



Danh sách kề (có hướng)

1: 2
2: 3
3: 1, 2, 4
4:

```
Adj[1].size() // 1  
Adj[2].size() // 1  
Adj[3].size() // 3  
Adj[4].size() // 0
```



1 Cơ bản về đồ thị

2 Tìm kiếm theo chiều sâu và ứng dụng - DFS

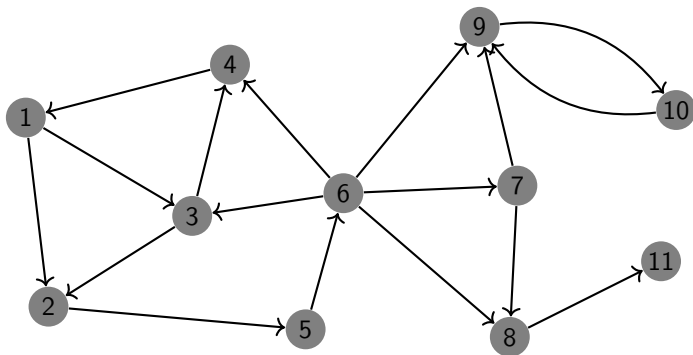
- DFS
- Thành phần liên thông
- Cây DFS
- Cầu
- TPLT mạnh
- Sắp xếp topo

3 Tìm kiếm theo chiều rộng và ứng dụng - BFS

Tìm kiếm theo chiều sâu - DFS

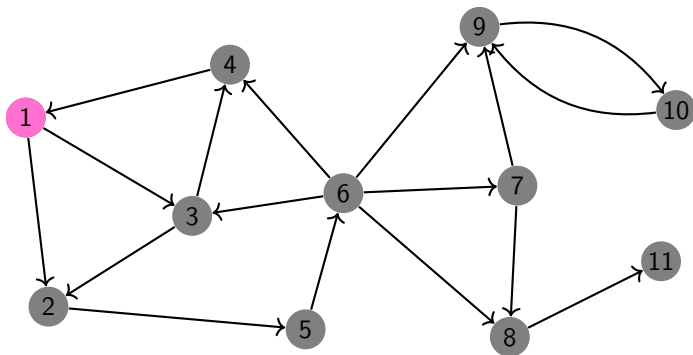
- Cho đồ thị $G = (V, E)$ (có hướng hoặc vô hướng) và hai đỉnh u và v , hỏi có tồn tại một đường đi từ u đến v ?
- Thuật toán tìm kiếm theo chiều sâu đưa ra được đường đi như vậy nếu tồn tại
- Thuật toán duyệt đồ thị ưu tiên theo chiều sâu (LIFO - Last In First Out), bắt đầu từ đỉnh xuất phát u
- Thực chất ta không cần chỉ rõ đỉnh v , vì ta có thể để thuật toán thăm tất cả các đỉnh có thể đến được từ u (mà vẫn cùng độ phức tạp)
- Vậy độ phức tạp của thuật toán là bao nhiêu?
- Mỗi đỉnh được thăm đúng một lần, và mỗi cạnh cũng được duyệt qua đúng một lần (nếu đến được)
- $\mathcal{O}(n + m)$

Tìm kiếm theo chiều sâu: Ví dụ



Stack:												
		1	2	3	4	5	6	7	8	9	10	11
Visited		F	F	F	F	F	F	F	F	F	F	F

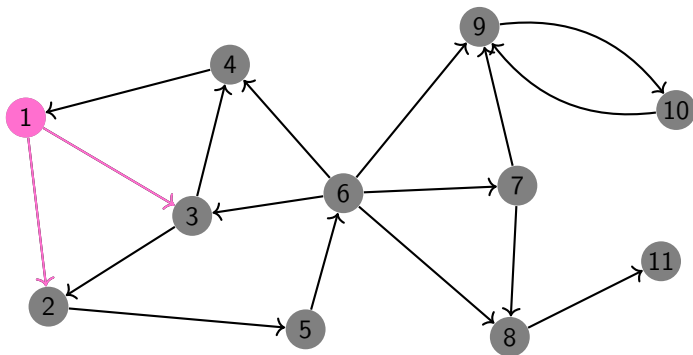
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 1 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	F	F	F	F	F	F	F	F	F	F

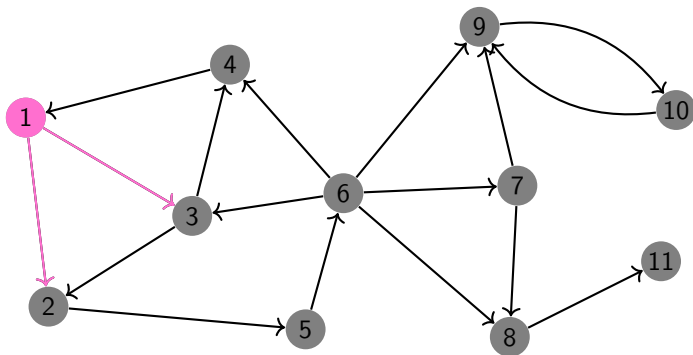
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 1 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	F	F	F	F	F	F	F	F	F	F

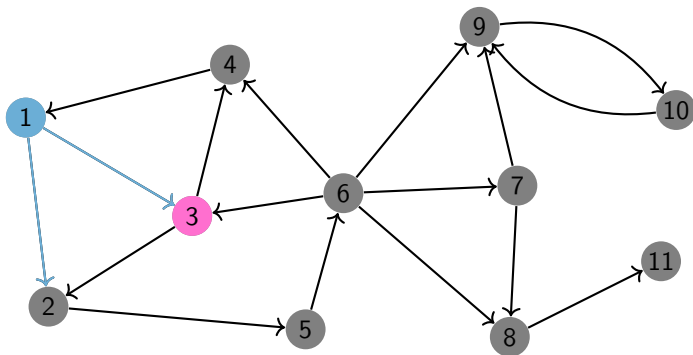
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 1 | 3 2

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	F	F	F	F	F	F	F	F

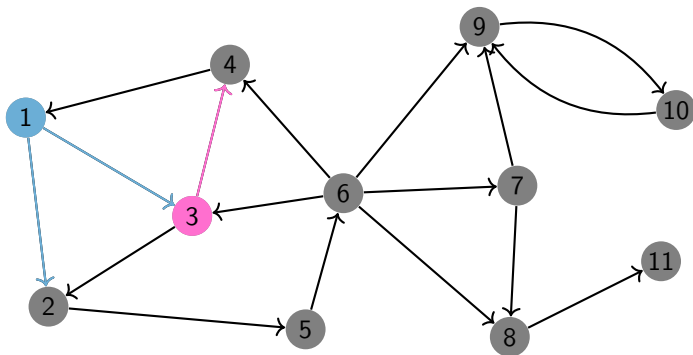
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 3 | 2

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	F	F	F	F	F	F	F	F

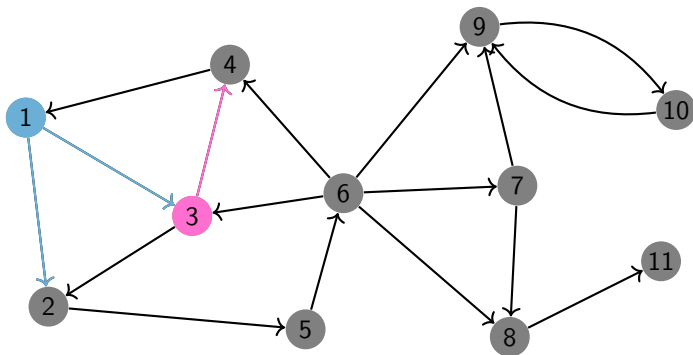
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 3 | 2

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	F	F	F	F	F	F	F	F

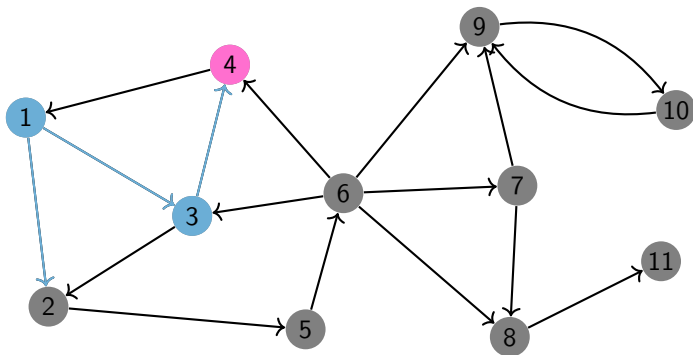
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 3 | 4 2

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	F	F	F	F	F	F	F

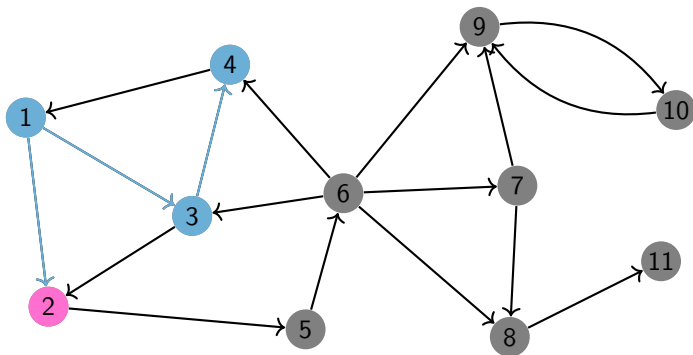
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 4 | 2

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	F	F	F	F	F	F	F

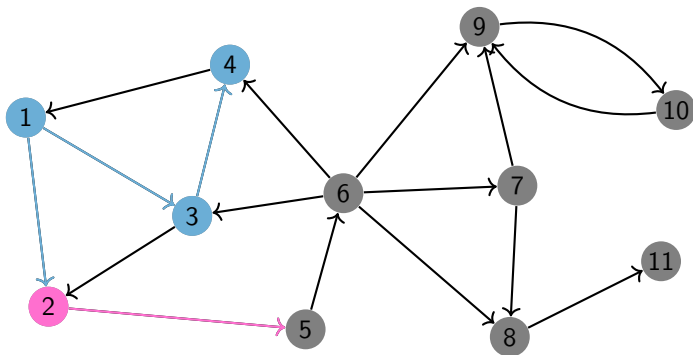
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 2 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	F	F	F	F	F	F	F

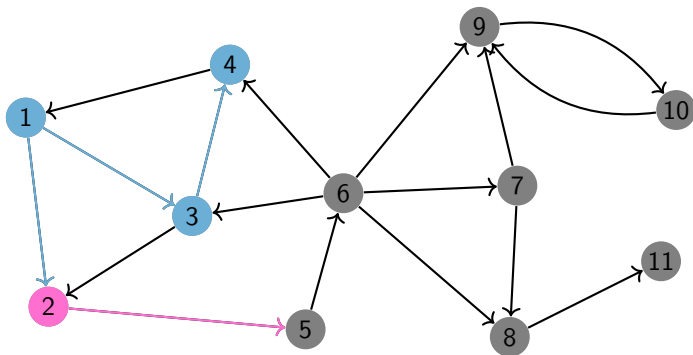
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 2 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	F	F	F	F	F	F	F

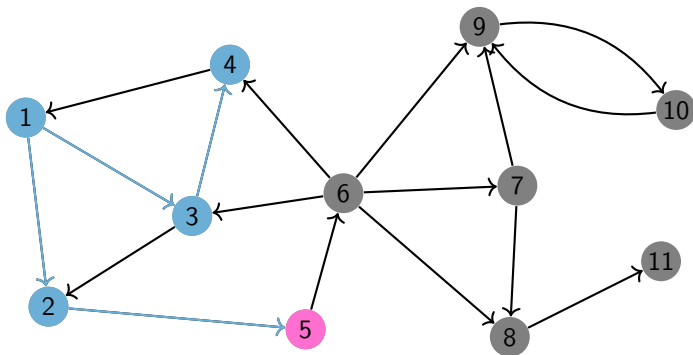
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 2 | 5

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	F	F	F	F	F	F

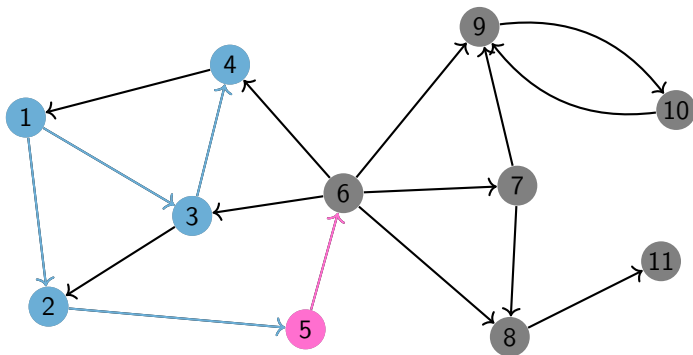
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 5 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	F	F	F	F	F	F

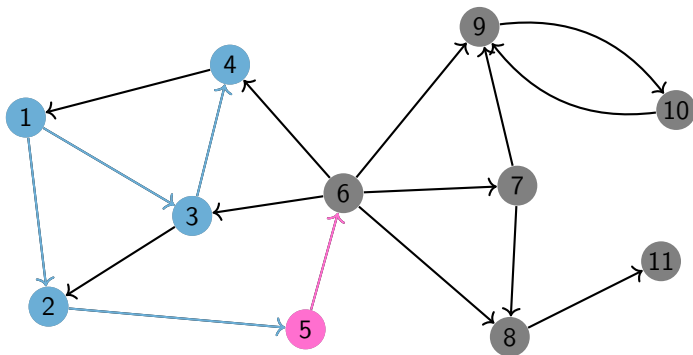
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 5 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	F	F	F	F	F	F

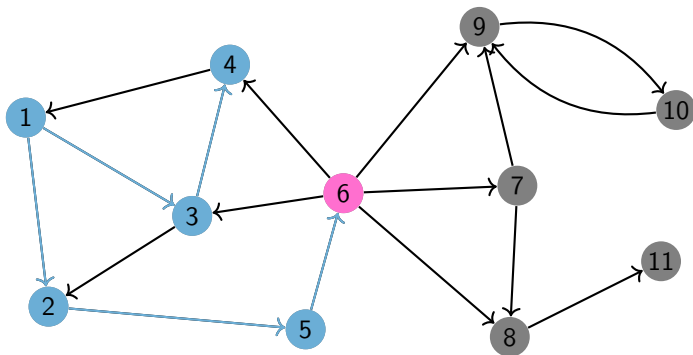
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 5 | 6

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	F	F	F	F	F

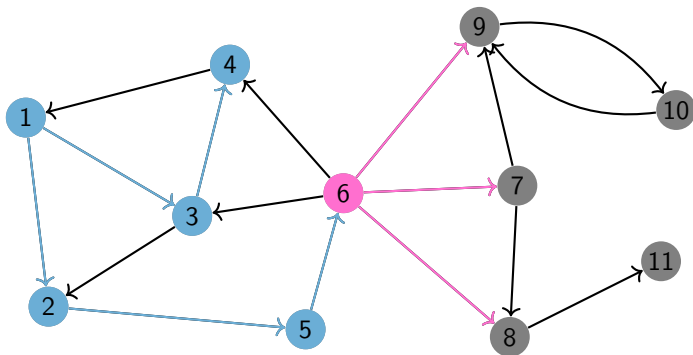
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 6 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	F	F	F	F	F

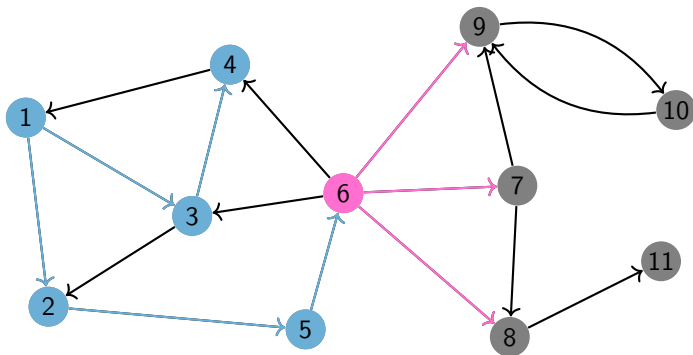
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 6 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	F	F	F	F	F

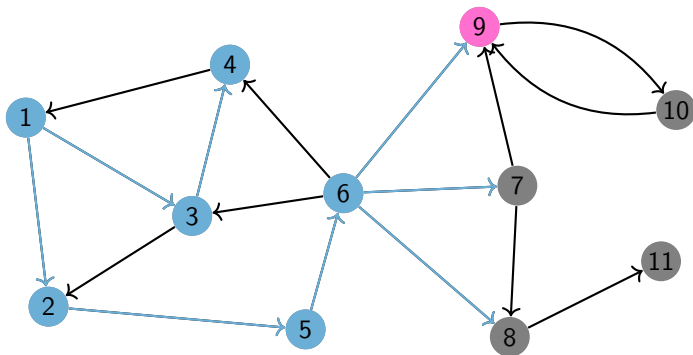
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 6 | 9 7 8

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	F	F

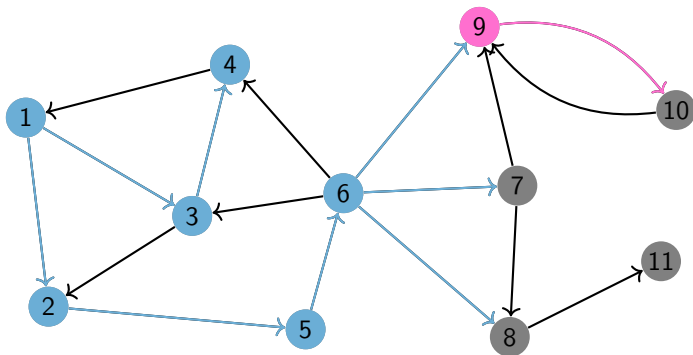
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 9 | 7 8

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	F	F

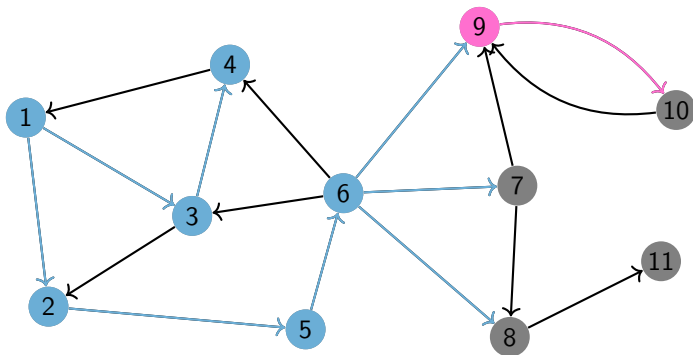
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 9 | 7 8

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	F	F

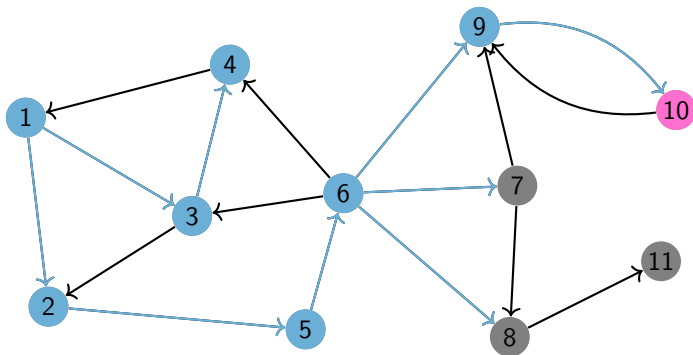
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 9 | 10 7 8

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	F

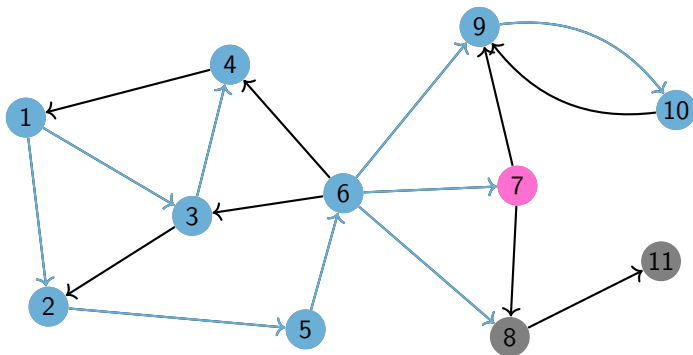
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 10 | 7 8

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	F

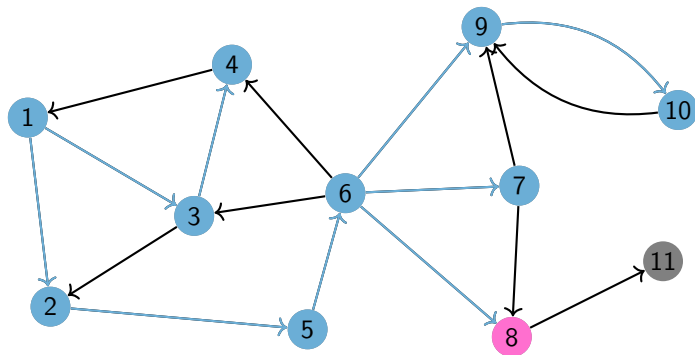
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 7 | 8

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	F

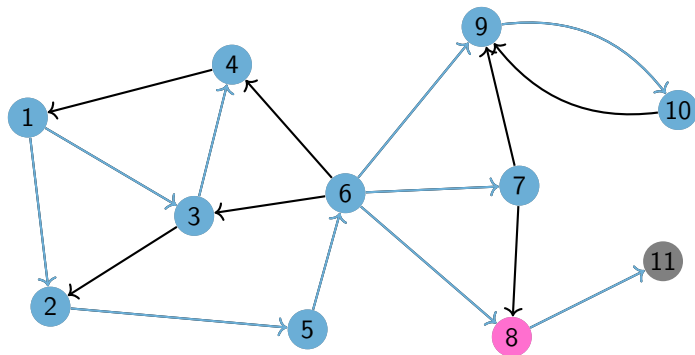
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 8 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	F

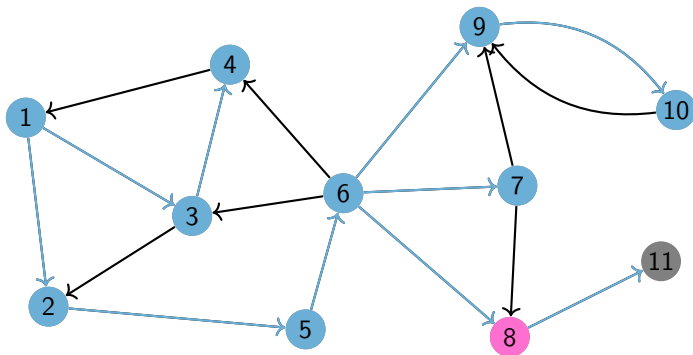
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 8 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	F

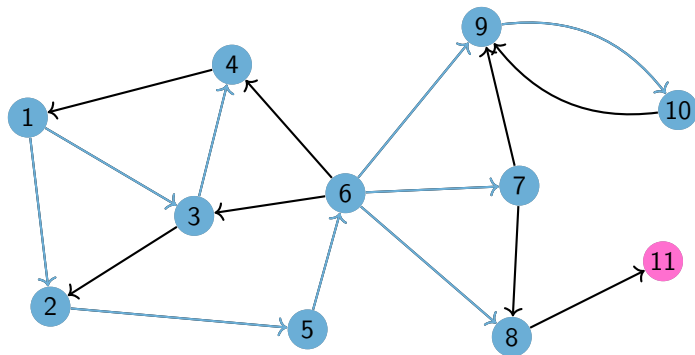
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 8 | 11

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	T

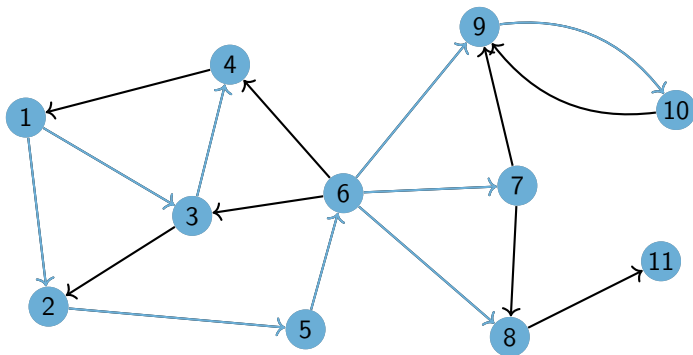
Tìm kiếm theo chiều sâu: Ví dụ



Stack: 11 |

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	T

Tìm kiếm theo chiều sâu: Ví dụ



Stack:												
		1	2	3	4	5	6	7	8	9	10	11
Visited		T	T	T	T	T	T	T	T	T	T	T

Tìm kiếm theo chiều sâu: Code

```
1  vector<int> Adj[1001];
2  vector<bool> bVisited(1001, false);
3
4  void DFS(int u) {
5      if (bVisited[u])
6          return;
7      bVisited[u] = true;
8      for (int i = 0; i < Adj[u].size(); ++i) {
9          int v = Adj[u][i];
10         bVisited[v] = true;
11         DFS(v);
12     }
13 }
```

1 Cơ bản về đồ thị

2 Tìm kiếm theo chiều sâu và ứng dụng - DFS

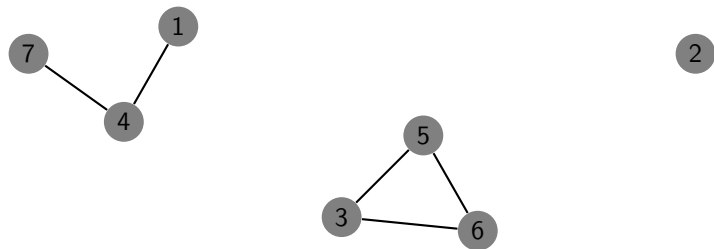
- DFS
- Thành phần liên thông
- Cây DFS
- Cầu
- TPLT mạnh
- Sắp xếp topo

3 Tìm kiếm theo chiều rộng và ứng dụng - BFS

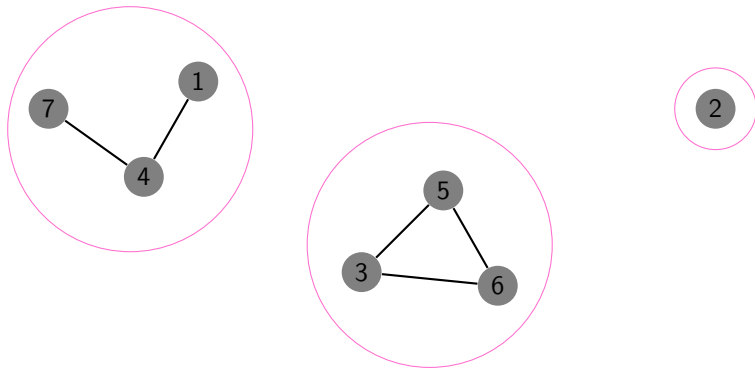
Thành phần liên thông: Bài toán

- Một đồ thị vô hướng có thể phân chia thành các thành phần liên thông (TPLT)
- Một TPLT là một tập con tối đa các đỉnh sao cho giữa hai đỉnh bất kỳ trong tập đều có đường đi giữa chúng
- Bài toán có thể được giải quyết bởi một số thuật toán cơ bản: Cấu trúc dữ liệu các tập không giao nhau (Union-Find), Tìm kiếm theo chiều sâu (DFS) hoặc Tìm kiếm theo chiều rộng (BFS)

Thành phần liên thông: Ví dụ



Thành phần liên thông: Ví dụ



Thành phần liên thông

- Cũng có thể sử dụng tìm kiếm theo chiều sâu để tìm các TPLT
- Lấy một đỉnh bất kỳ và gọi thuật toán tìm kiếm theo chiều sâu xuất phát từ đỉnh đó
- Tất cả các đỉnh đến được từ đỉnh xuất phát đó đều thuộc cùng một TPLT
- Lặp lại quá trình trên đến khi thu được toàn bộ các TPLT
- Độ phức tạp $\mathcal{O}(n + m)$

Thành phần liên thông: Code

```
1  vector<int> Adj[1001];
2  vector<int> iComponent(1001, -1);
3
4  void Find_Component(int cur_comp, int u) {
5      if (iComponent[u] != -1) return;
6
7      iComponent[u] = cur_comp;
8
9      for (int i = 0; i < Adj[u].size(); ++i) {
10         int v = Adj[u][i];
11         Find_Component(cur_comp, v);
12     }
13 }
14 int num_comp = 0;
15 for (int u = 1; u <= n; ++u) {
16     if (iComponent[u] == -1) {
17         Find_Component(num_comp, u);
18         num_comp++;
19     }
20 }
```

1 Cơ bản về đồ thị

2 Tìm kiếm theo chiều sâu và ứng dụng - DFS

- DFS
- Thành phần liên thông
- Cây DFS
- Cầu
- TPLT mạnh
- Sắp xếp topo

3 Tìm kiếm theo chiều rộng và ứng dụng - BFS

Cây DFS: Giới thiệu

- Khi gọi DFS từ một đỉnh nào đó, các vết tìm kiếm tạo thành một cây
- Khi duyệt từ một đỉnh gặp một đỉnh khác chưa được thăm, cạnh duyệt qua đó gọi là *cạnh xuôi* (*forward edge*)
- Khi duyệt từ một đỉnh gặp một đỉnh là tổ tiên của nó (đã thăm trước đó trong cùng quá trình duyệt hiện tại), cạnh duyệt qua đó gọi là *cạnh ngược* (*backward edge*)
- Ngoài ra còn khái niệm cạnh *cạnh vòng* (*cross edge*) trên đồ thị có hướng đối với những loại cạnh còn lại
- Tập các cạnh xuôi tạo thành một cây DFS
- *xem ví dụ*

Cây DFS: Giới thiệu

- Cây từ các cạnh xuôi, cùng với các cạnh ngược, chứa đựng rất nhiều thông tin về đồ thị ban đầu
- Ví dụ: một cạnh ngược luôn nằm trên một chu trình của đồ thị ban đầu
- Nếu không có cạnh ngược thì không có chu trình trong đồ thị ban đầu (nghĩa là loại đồ thị không có chu trình - acyclic graph)

Phân tích cây DFS

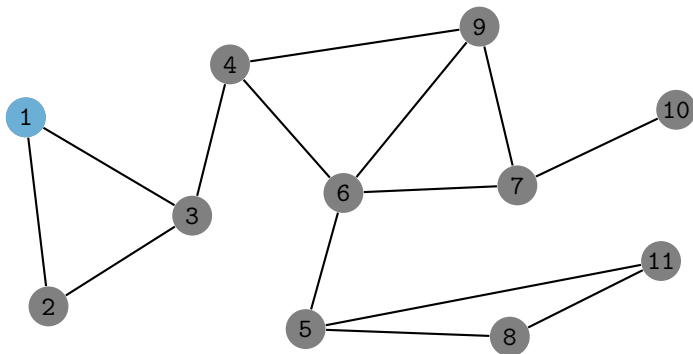
- Hãy quan sát kỹ hơn cây DFS
- Đầu tiên, đánh số các đỉnh theo thứ tự duyệt của thuật toán DFS trong mảng `num`
- Với mỗi đỉnh `u`, cần tính `Low[u]` là chỉ số của đỉnh nhỏ nhất mà `u` có thể chạm được (bởi một cạnh ngược) khi tiếp tục duyệt theo cây con có gốc tại `u`
- Khởi tạo thì `Low[u] = Num[u]`, `Low[u]` sẽ thay đổi khi có một cạnh ngược.
- Những thông số này rất hữu ích cho các bài toán quan trọng: tìm khớp/cầu, tìm thành phần liên thông mạnh, ...

Phân tích cây DFS: Code

```
1  const int n = 1001;
2  vector<int> Adj[n];
3  vector<int> Low(n), Num(n, -1);
4  int curnum = 0;
5  void AnalyzeDFS(int u, int p) {
6      Low[u] = Num[u] = ++curnum;
7      for (int i = 0; i < Adj[u].size(); ++i) {
8          int v = Adj[u][i];
9          if (v == p) continue; //Khong_Xet_Dinh_Ngay_Truoc
10         if (Num[v] == -1) {
11             AnalyzeDFS(v, u);
12             Low[u] = min(Low[u], Low[v]);
13         } else {
14             Low[u] = min(Low[u], Num[v]);
15         }
16     }
17 }
```

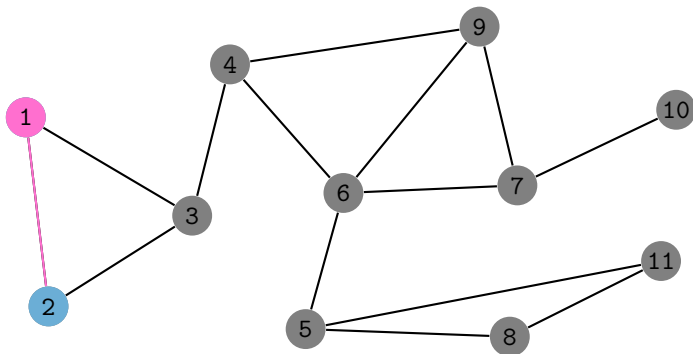
- Gọi AnalyzeDFS(u, -1) với mọi u chưa được thăm (Num[u] == -1)

Phân tích cây DFS: Ví dụ



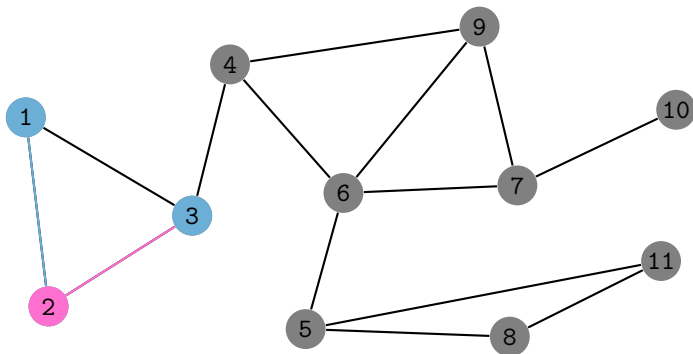
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



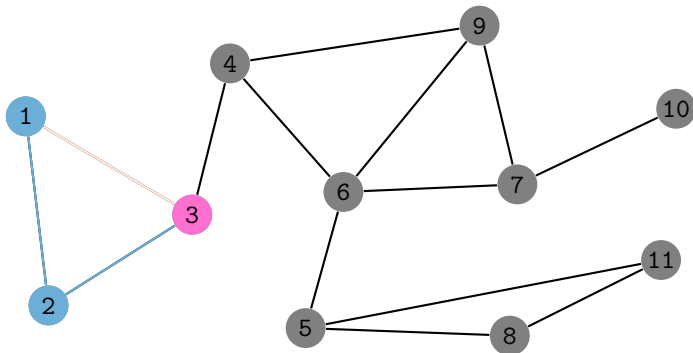
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



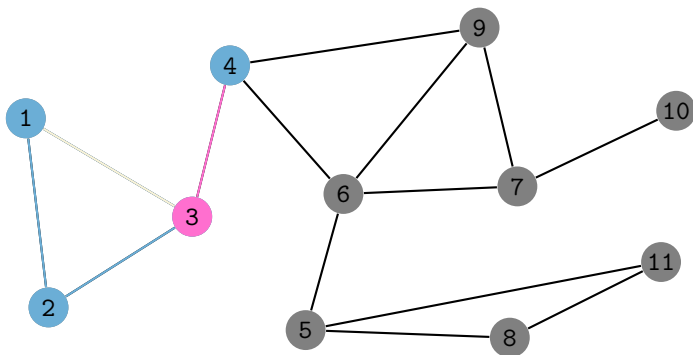
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



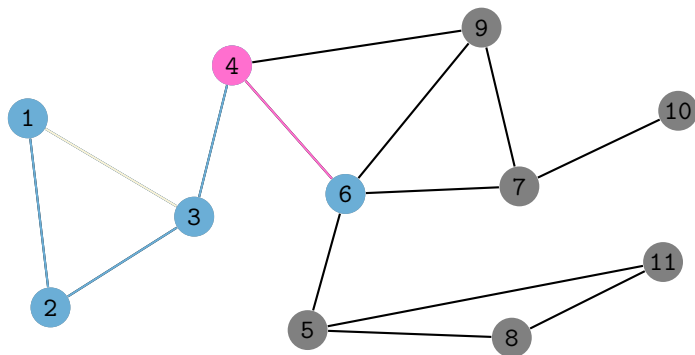
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



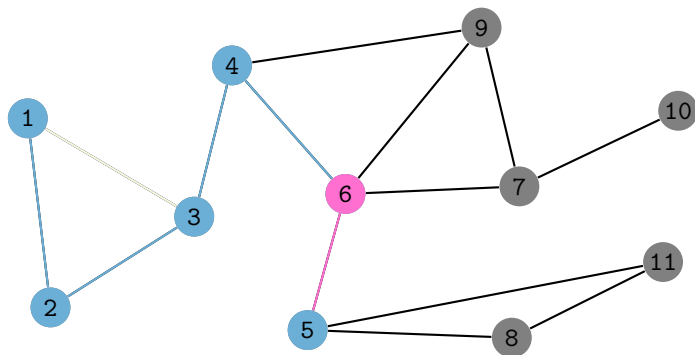
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



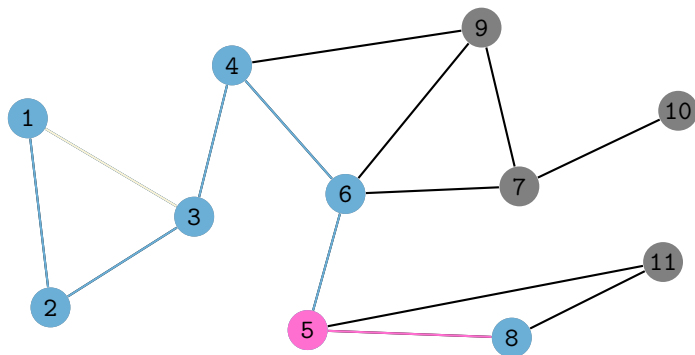
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



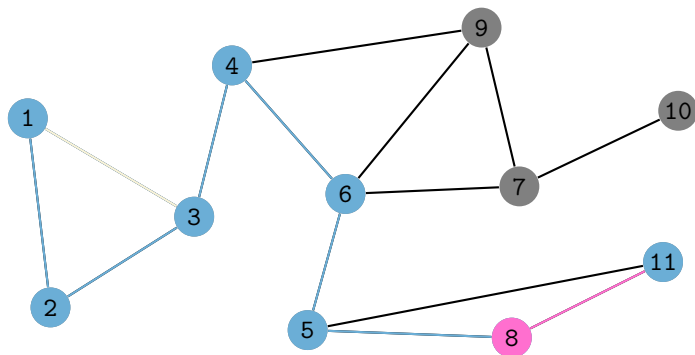
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



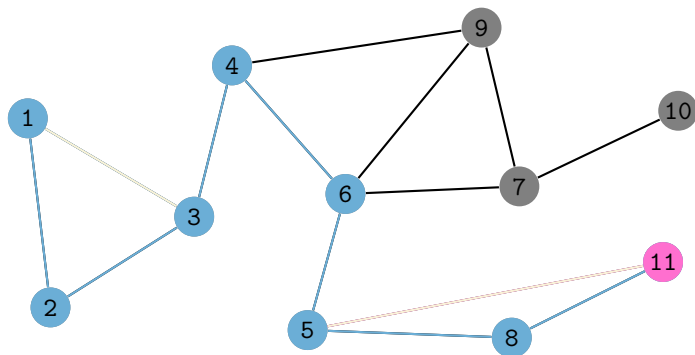
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



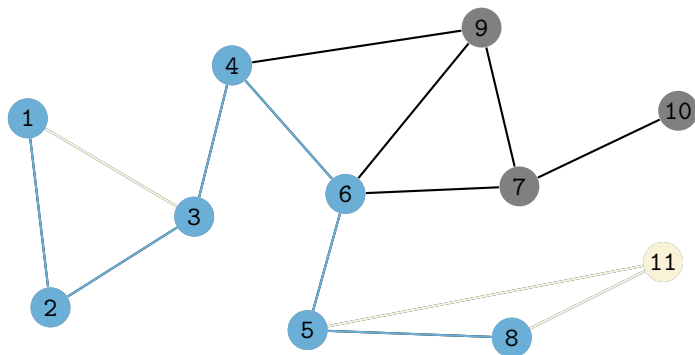
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



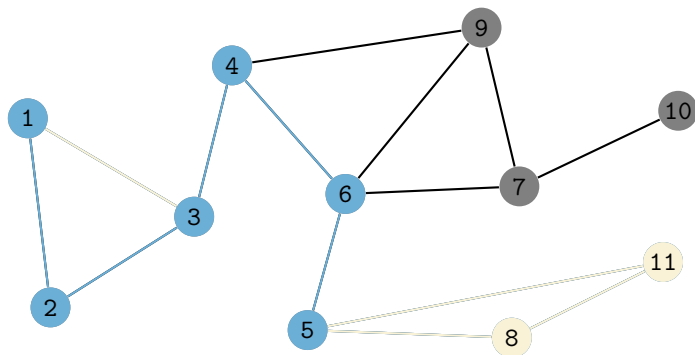
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



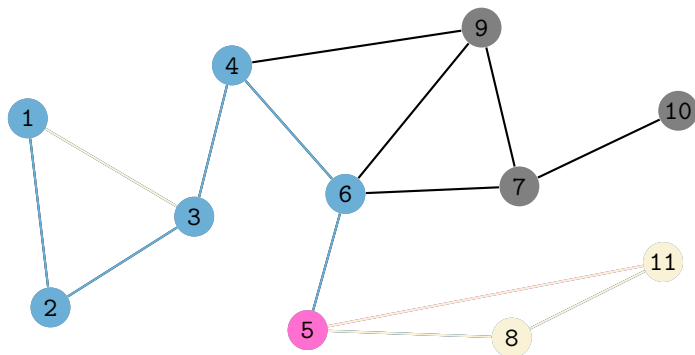
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



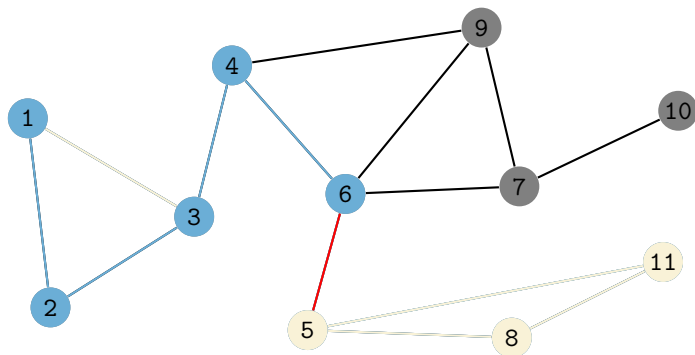
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



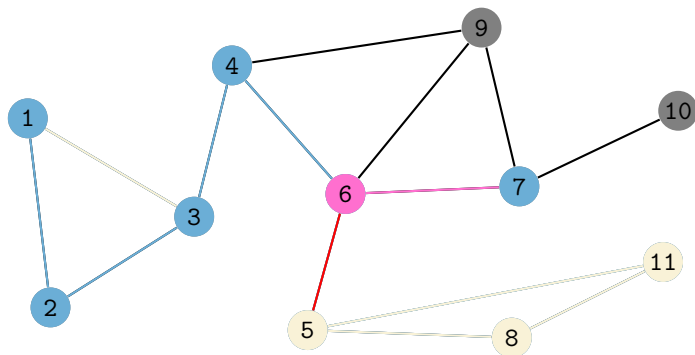
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



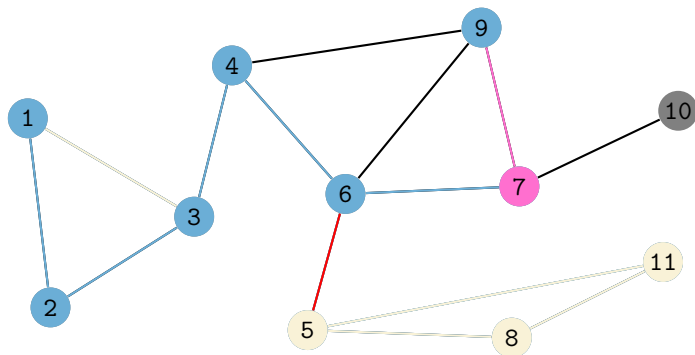
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



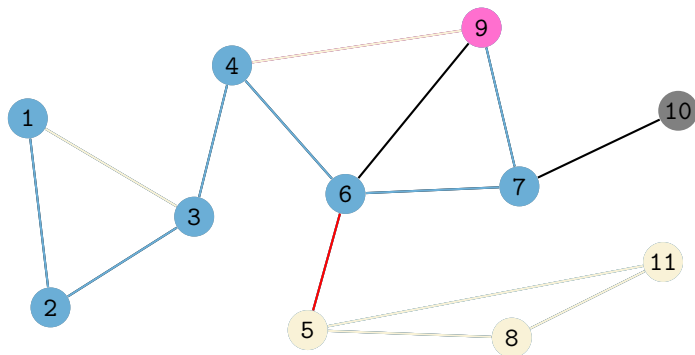
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



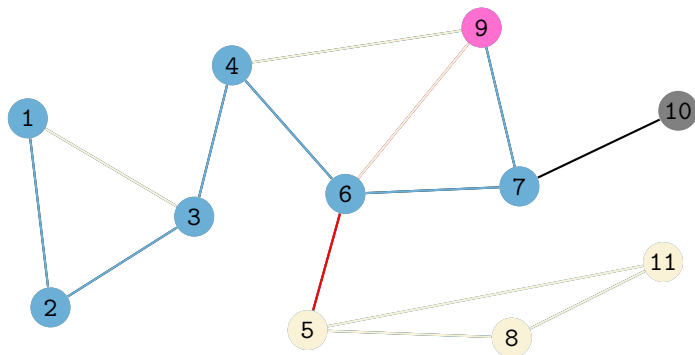
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



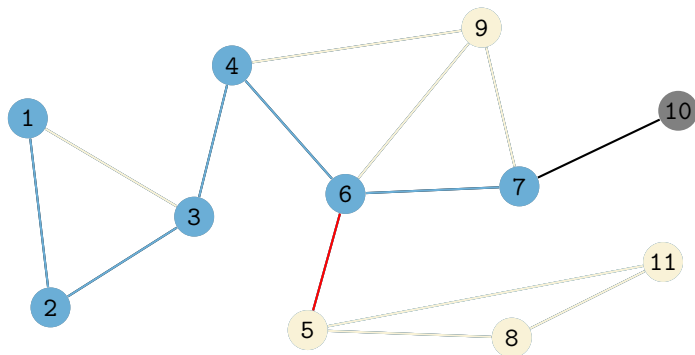
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



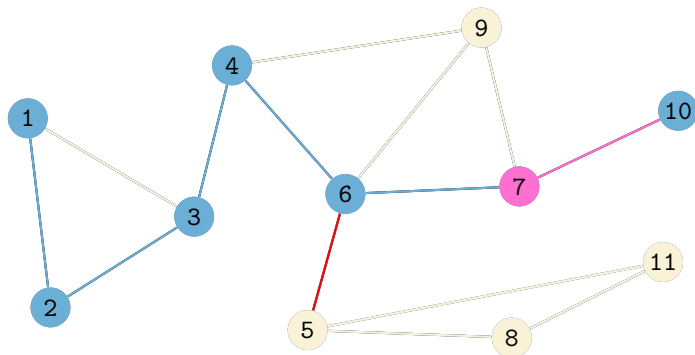
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



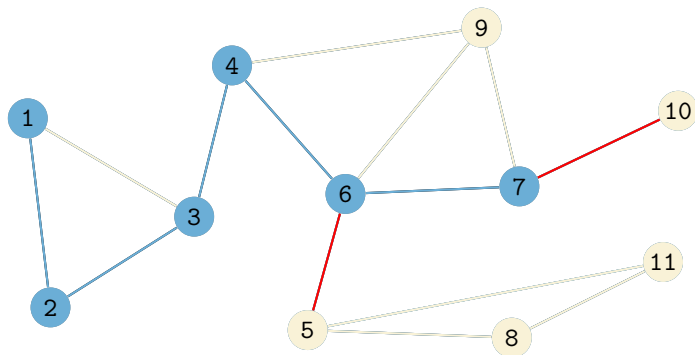
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



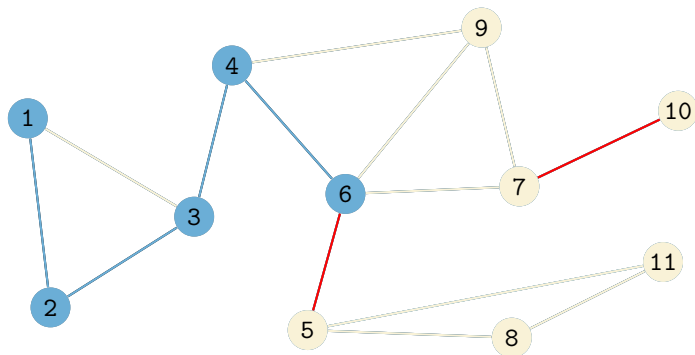
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



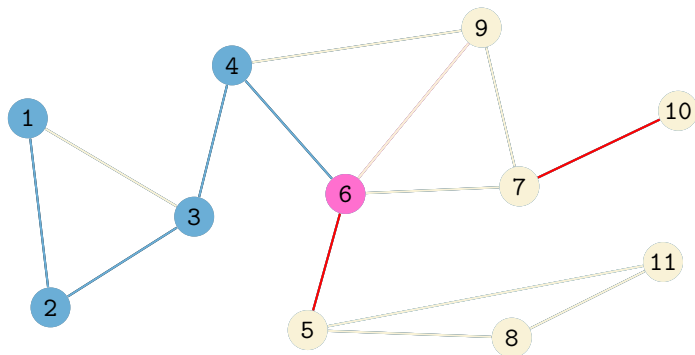
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



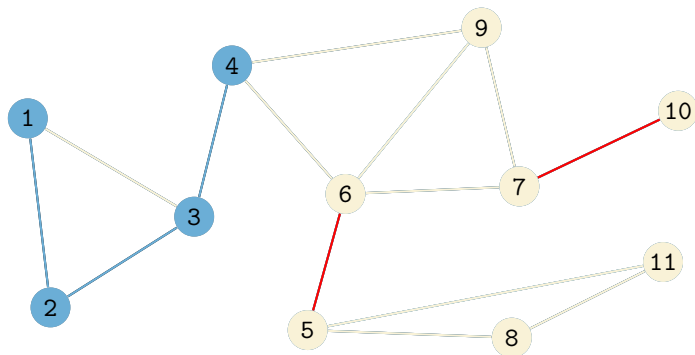
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



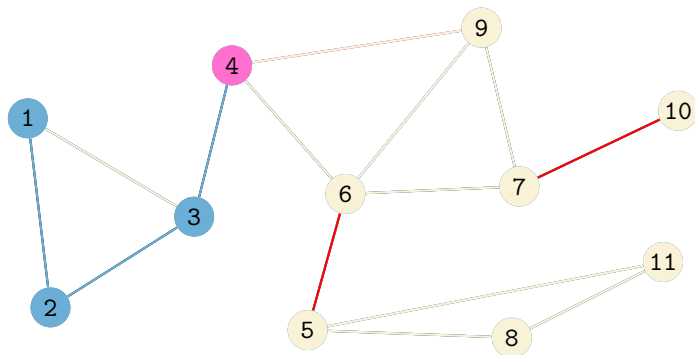
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



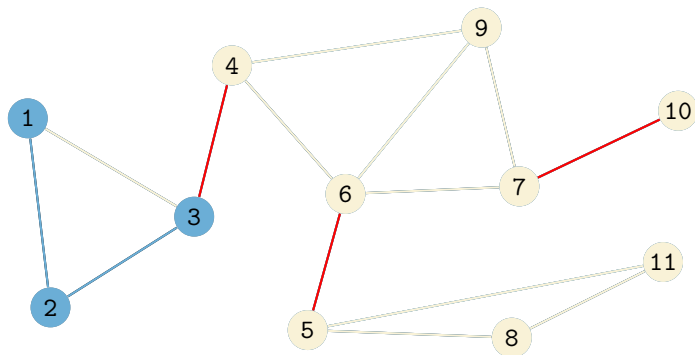
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



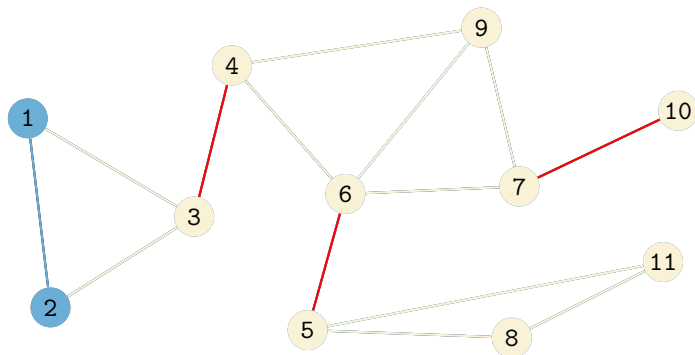
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



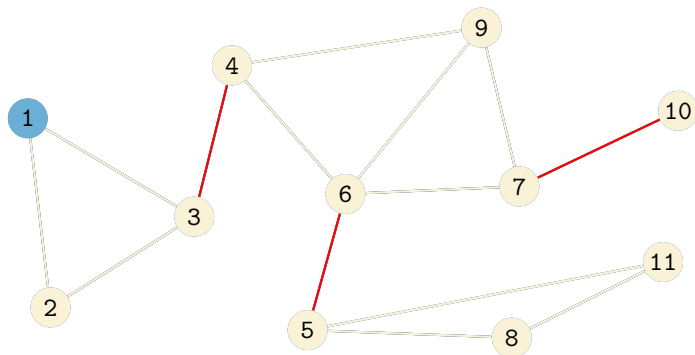
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



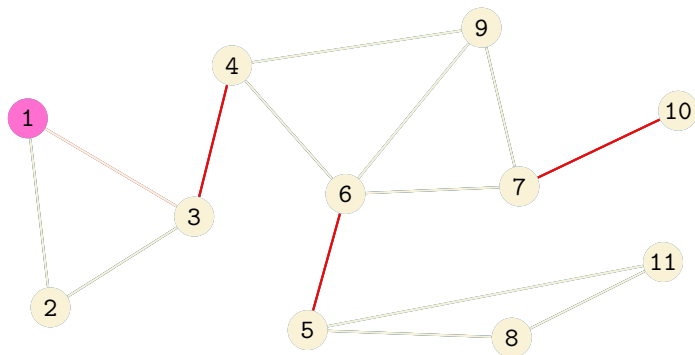
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



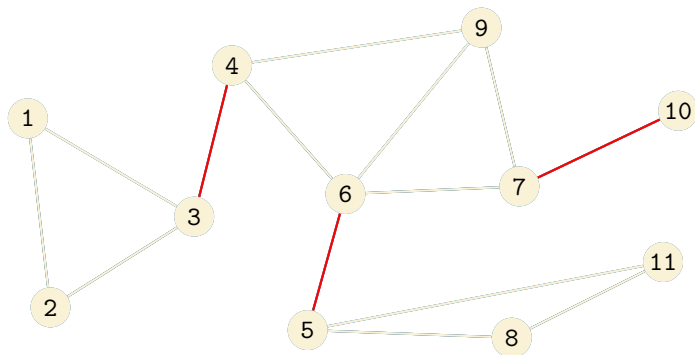
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS: Ví dụ



i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Phân tích cây DFS

- Độ phức tạp chỉ là $\mathcal{O}(n + m)$, do chỉ gọi một lần DFS
- Bây giờ hãy xem một số ứng dụng của thuật toán này

1 Cơ bản về đồ thị

2 Tìm kiếm theo chiều sâu và ứng dụng - DFS

- DFS
- Thành phần liên thông
- Cây DFS
- Cầu
- TPLT mạnh
- Sắp xếp topo

3 Tìm kiếm theo chiều rộng và ứng dụng - BFS

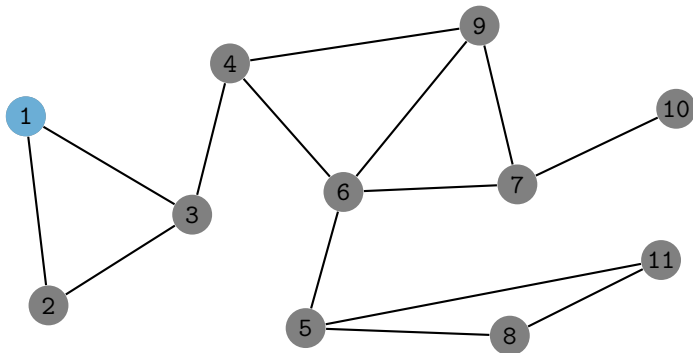
Câu: Giới thiệu

- Cho đồ thị không trọng số $G = (V, E)$
- Không mất tính tổng quát, giả sử G liên thông (nghĩa là G là một TPLT lớn)
- Tìm một cạnh mà nếu loại bỏ cạnh đó ra khỏi G thì G mất tính liên thông
- Thuật toán trực tiếp: Thử loại bỏ từng cạnh một, và tính số TPLT thu được
- Cách này thiếu hiệu quả: $\mathcal{O}(m(n + m))$

Cầu: Giới thiệu

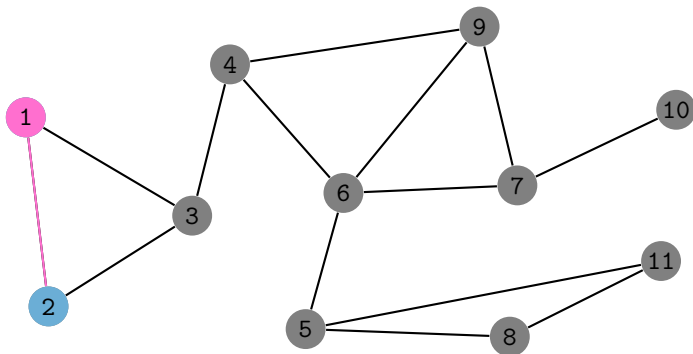
- Bây giờ quan sát các giá trị tính được từ cây DFS
- Nhận thấy rằng một cạnh xuôi (u, v) là cầu khi và chỉ khi $\text{Low}[v] > \text{Num}[u]$
- Như vậy chỉ cần mở rộng thuật toán phân tích cây DFS ở trên để đưa ra toàn bộ cầu
- Độ phức tạp thuật toán chỉ là $\mathcal{O}(n + m)$ với chỉ một lần gọi DFS đối với mỗi TPLT!

Cầu: Ví dụ



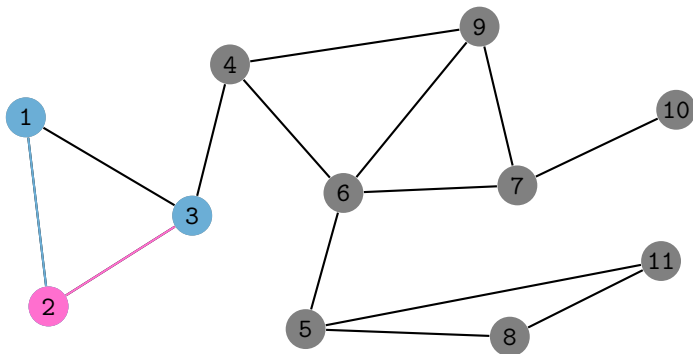
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



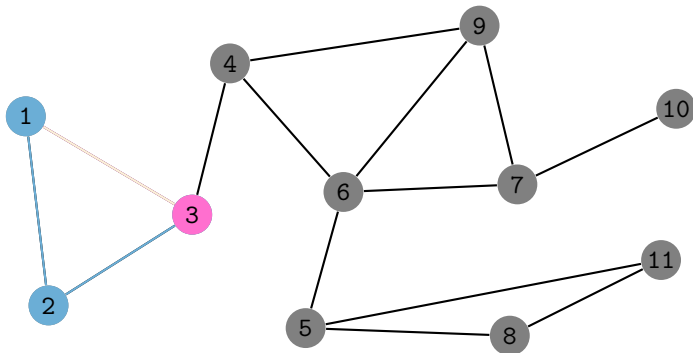
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



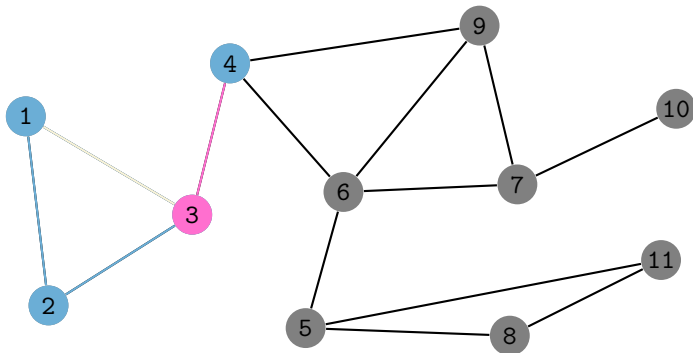
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



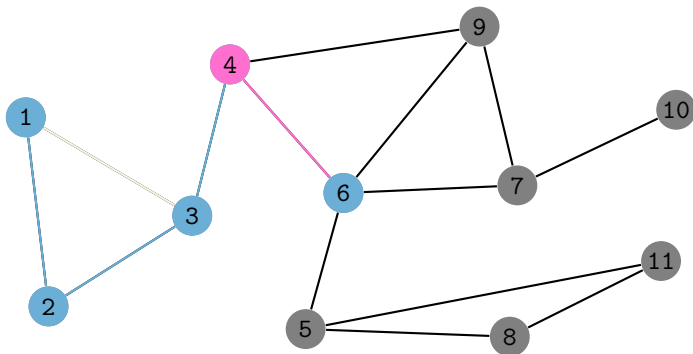
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



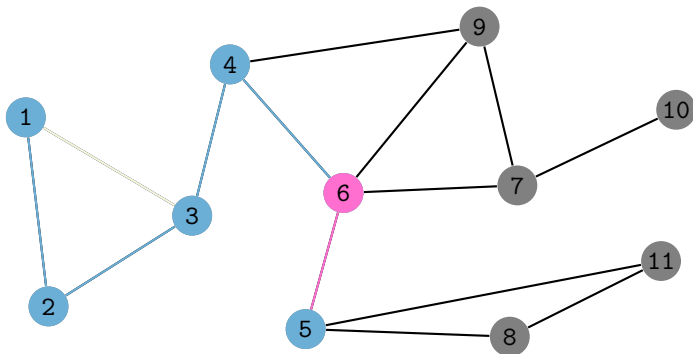
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



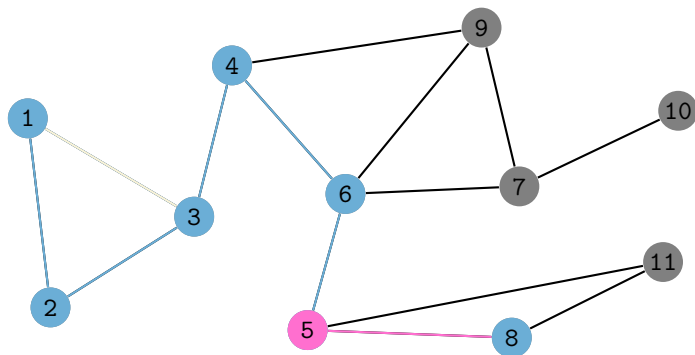
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



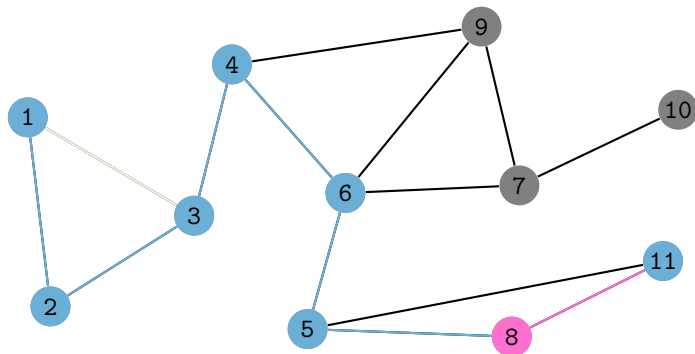
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



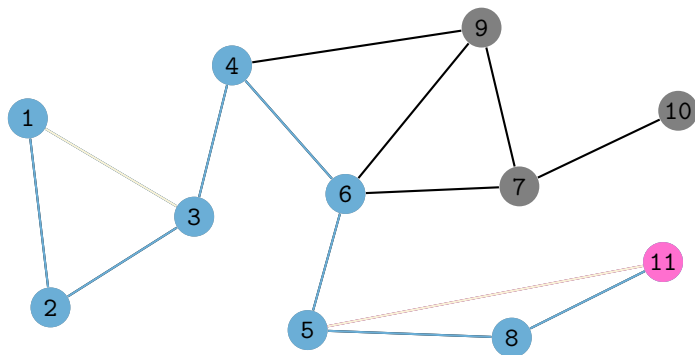
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



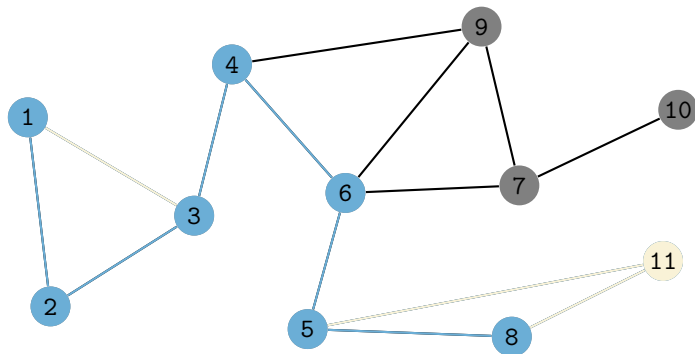
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



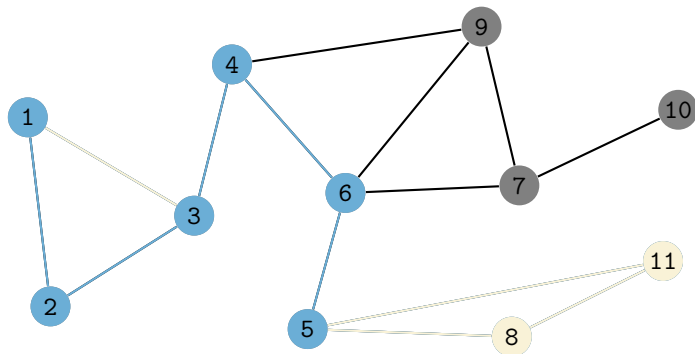
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



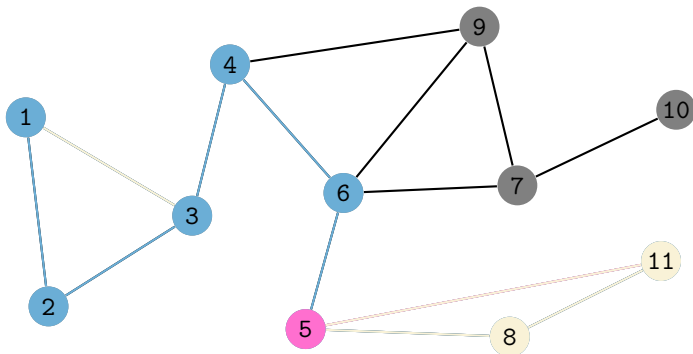
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



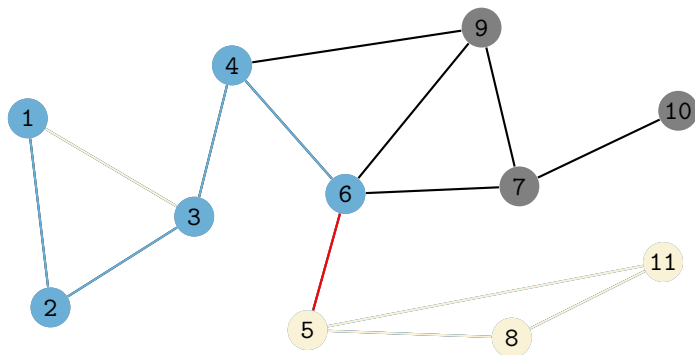
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



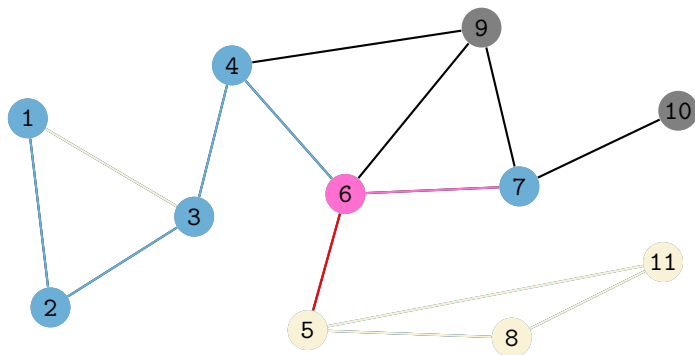
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



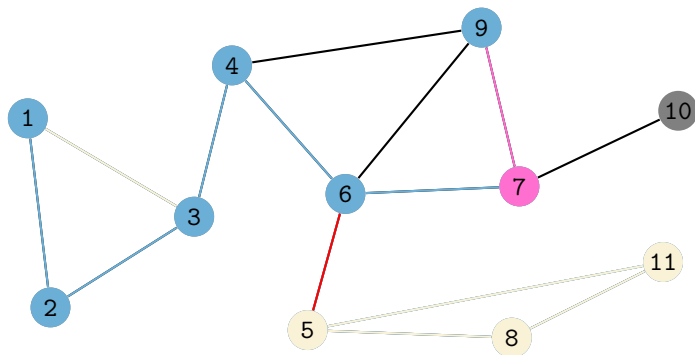
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



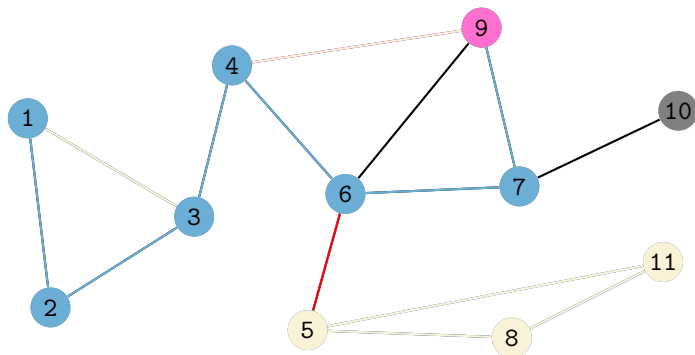
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



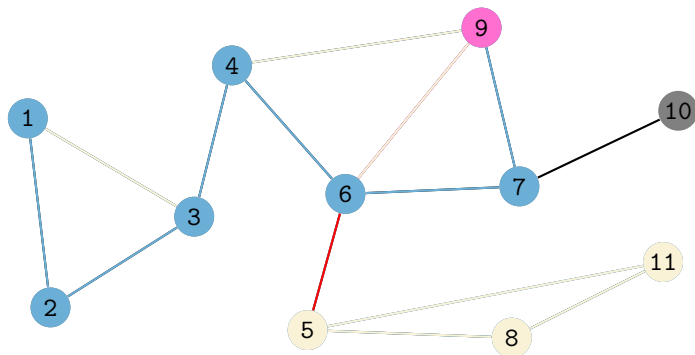
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



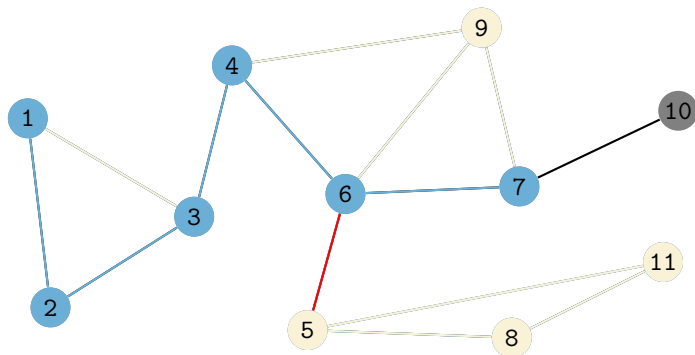
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



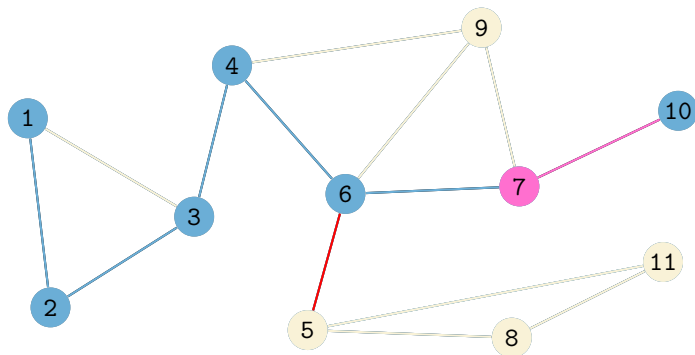
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



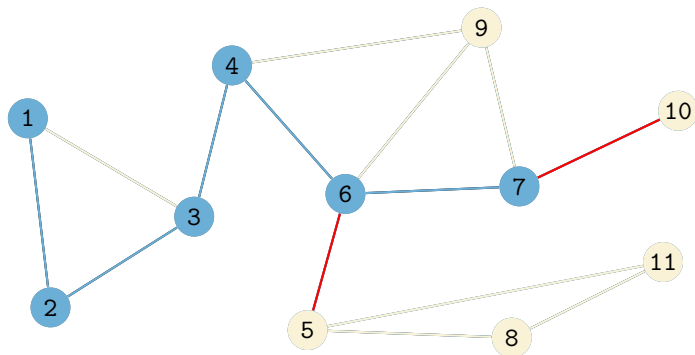
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



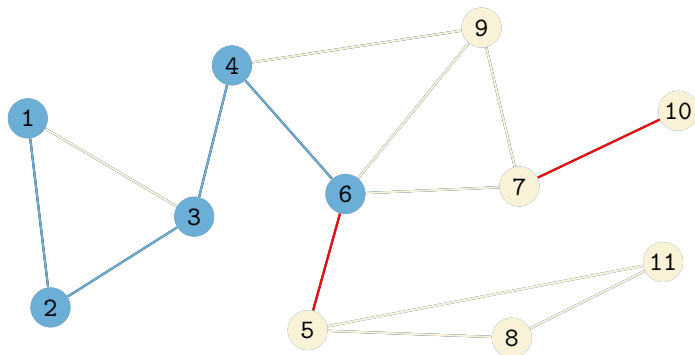
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



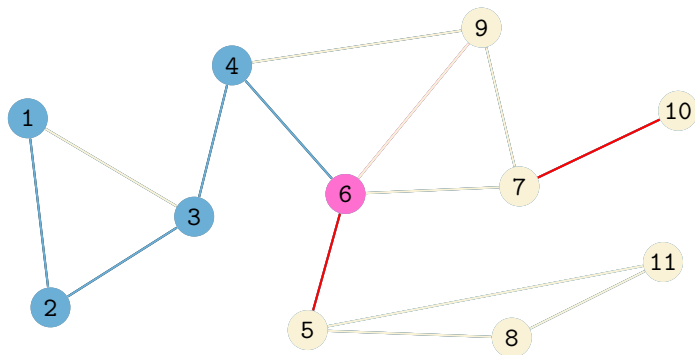
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



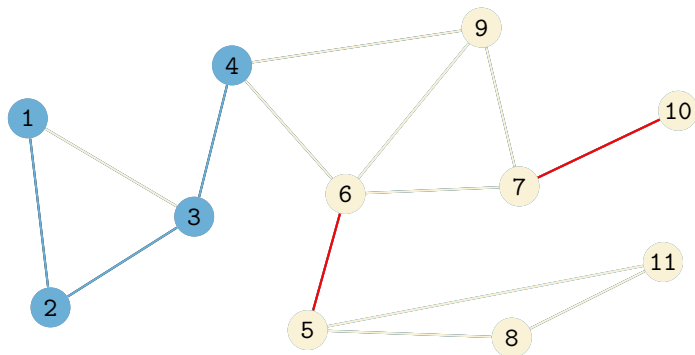
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



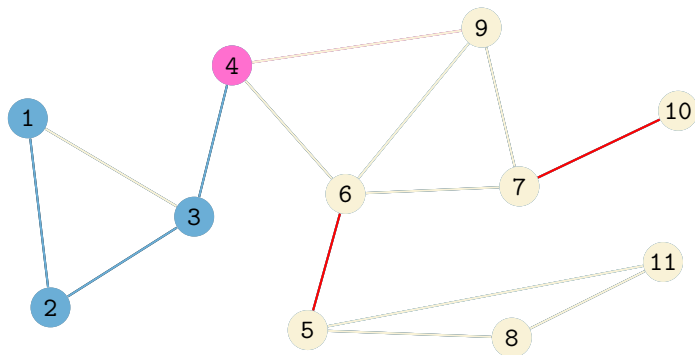
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



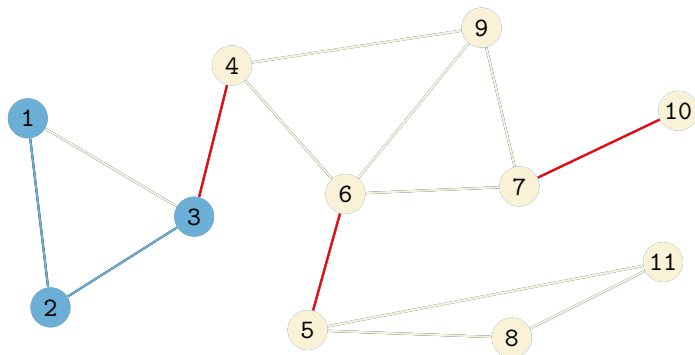
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



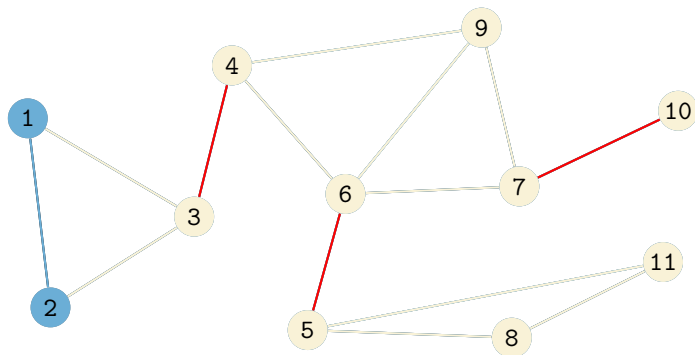
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



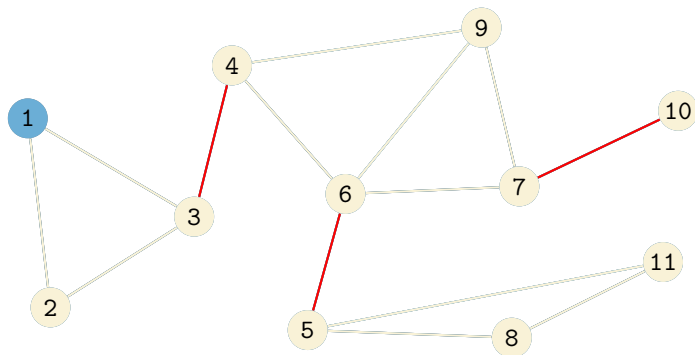
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



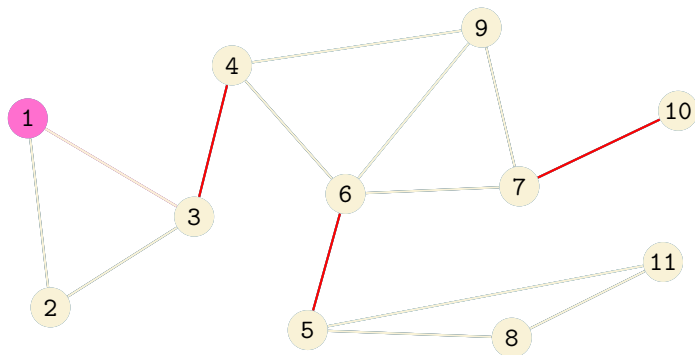
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



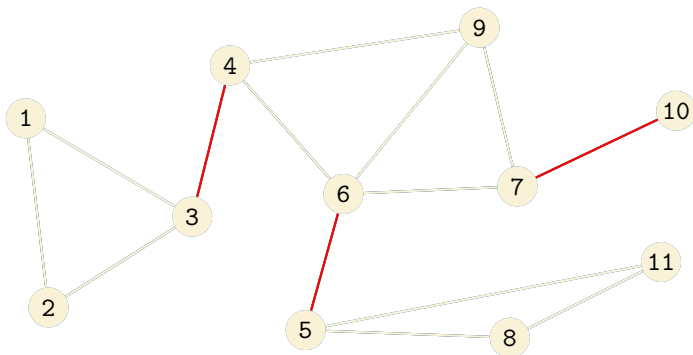
i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Cầu: Ví dụ



i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

Câu: Code

```
1  const int n = 1001;
2  vector<int> Adj[n], Low(n), Num(n, -1);
3  int curnum = 0;
4  vector<pair<int, int> > iiBridges;
5
6  void Find_Bridges(int u, int p) {
7      Low[u] = Num[u] = ++curnum;
8      for (int i = 0; i < Adj[u].size(); ++i) {
9          int v = Adj[u][i];
10         if (v == p) continue;
11         if (Num[v] == -1) {
12             Find_Bridges(v, u);
13             Low[u] = min(Low[u], Low[v]);
14         } else {
15             Low[u] = min(Low[u], Num[v]);
16         }
17         if (Low[v] > Num[u])
18             iiBridges.push_back(make_pair(u, v));
19     }
20 }
```

- Gọi Find_Bridges(u, -1) với mọi u chưa được thăm

1 Cơ bản về đồ thị

2 Tìm kiếm theo chiều sâu và ứng dụng - DFS

- DFS
- Thành phần liên thông
- Cây DFS
- Cầu
- TPLT mạnh
- Sắp xếp topo

3 Tìm kiếm theo chiều rộng và ứng dụng - BFS

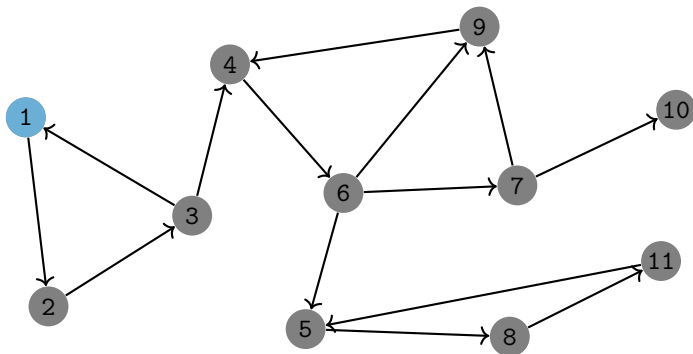
TPLT mạnh: Giới thiệu

- Ta đã biết cách tìm các TPLT trên đồ thị vô hướng
- Thế trên đồ thị có hướng thì sao?
- Các TPLT này có một chút khác biệt trên đồ thị có hướng, bởi vì nếu v đến được từ u thì không có nghĩa là u đến được từ v
- Tuy vậy, định nghĩa vẫn tương tự
- Một TPLT mạnh là một tập con tối đa các đỉnh sao cho giữa hai đỉnh bất kỳ trong tập luôn có đường đi từ đỉnh này đến đỉnh kia và ngược lại

TPLT mạnh: Giới thiệu

- Thuật toán tìm các TPLT ở trên không áp dụng được
- Thay vào đó ta có thể sử dụng cây DFS để tìm các TPLT mạnh này
- *xem ví dụ*

TPLT mạnh: Ví dụ



i	1	2	3	4	5	6	7	8	9	10	11
Num[i]	1	2	3	4	6	5	9	7	10	11	8
Low[i]	1	1	1	4	6	4	4	6	4	11	6

TPLT mạnh

- Sau quá trình phân tích cây tại đỉnh u mà $Low[u] = Num[u]$ thì ta tìm được một thành phần liên thông mạnh theo quá trình duyệt cây từ u
- Sử dụng mảng `bConnect[]` dùng để kiểm tra xem đỉnh v có còn được “kết nối” trong đồ thị hay không ? Nếu phát hiện ra một thành phần liên thông mạnh, và một đỉnh v có trong thành phần liên thông mạnh đó, thì ta loại đỉnh v này ra khỏi đồ thị bằng câu lệnh `bConnect[v] = 0`, điều này là quan trọng vì để tránh gây ảnh hưởng đến việc tính mảng `Low[]` của những đỉnh khác vẫn còn nằm trong đồ thị

TPLT mạnh: Code

```
1  vector<int> Adj[10001];
2  vector<int> Low(10001), Num(10001, -1);
3  vector<bool> bConnect(10001, false);
4  int curnum = 0;
5
6  stack<int> iComp;
7
8  void SCC(int u) {
9      // SCC code...
10 }
11
12 int main() {
13     for (int i = 0; i < n; ++i)
14         if (Num[i] == -1)
15             SCC(i);
16
17     return;
18 }
```

TPLT mạnh: Code

```
1 void SCC(int u) {
2     iComp.push(u);
3     bConnect[u] = true;
4     Low[u] = Num[u] = ++curnum;
5
6     for (int i = 0; i < Adj[u].size(); ++i) {
7         int v = Adj[u][i];
8         if (Num[v] == -1) {
9             SCC(v);
10            Low[u] = min(Low[u], Low[v]);
11        } else if (iConnect[v]) {
12            Low[u] = min(Low[u], Num[v]);
13        }
14    }
15
16    if (Num[u] == Low[u]) {
17        cout << "TPLT: ";
18        while (true) {
19            int cur = iComp.top();
20            iComp.pop();
21            bConnect[cur] = false;
22            cout << cur << " ";
23            if (cur == u) break;
24        }
25        cout << endl;
26    }
27 }
```

TPLT mạnh

- Độ phức tạp thuật toán?
- Cơ bản là chỉ dùng thuật toán phân tích cây DFS (có độ phức tạp $\mathcal{O}(n + m)$), cộng thêm một vòng lặp để xây dựng TPLT mạnh
- Do mỗi đỉnh chỉ thuộc một TPLT, độ phức tạp vẫn chỉ là $\mathcal{O}(n + m)$
- Thuật toán được biết đến với tên gọi thuật toán Tarjan

1 Cơ bản về đồ thị

2 Tìm kiếm theo chiều sâu và ứng dụng - DFS

- DFS
- Thành phần liên thông
- Cây DFS
- Cầu
- TPLT mạnh
- Sắp xếp topo

3 Tìm kiếm theo chiều rộng và ứng dụng - BFS

Sắp xếp topo: Bài toán

- Có n công việc
- Mỗi công việc i có một danh sách các công việc cần phải hoàn thành trước khi bắt đầu công việc i
- Hãy tìm một trình tự mà ta có thể thực hiện toàn bộ các công việc
- Có thể biểu diễn trên một đồ thị có hướng
 - ▶ Mỗi công việc là một đỉnh của đồ thị
 - ▶ Nếu công việc j phải hoàn thành trước công việc i , thì thêm một cạnh có hướng từ đỉnh i đến đỉnh j
- Lưu ý là không thể có một trình tự thỏa mãn nếu như đồ thị có chu trình
- Có thể sửa đổi thuật toán DFS để đưa ra một trình tự thỏa mãn trong thời gian $\mathcal{O}(n + m)$, hoặc đưa ra không có trình tự thỏa mãn

Sắp xếp topo:code

```
1  vector<int> Adj[1001];
2  vector<bool> bVisited(1001, false);
3  vector<int> iOrder;
4
5  void Topo_Sort(int u) {
6      if (bVisited[u]) return;
7
8      bVisited[u] = true;
9      for (int i = 0; i < Adj[u].size(); ++i) {
10         int v = Adj[u][i];
11         Topo_Sort(v);
12     }
13     iOrder.push_back(u);
14 }
```

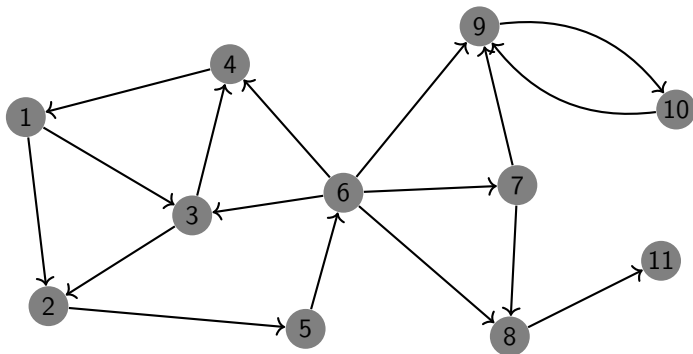
- Gọi Topo_Sort(u) với mọi đỉnh u chưa được thăm

- 1 Cơ bản về đồ thị
- 2 Tìm kiếm theo chiều sâu và ứng dụng - DFS
- 3 Tìm kiếm theo chiều rộng và ứng dụng - BFS
 - Tìm kiếm theo chiều rộng - BFS
 - Đường đi ngắn nhất trên đồ thị không trọng số

Tìm kiếm theo chiều rộng - BFS

- Thuật toán tìm kiếm theo chiều rộng chỉ khác DFS ở trình tự thăm các đỉnh
- BFS thăm đỉnh theo trình tự FIFO (First In First Out)
- BFS cho trình tự thăm tăng dần theo khoảng cách từ đỉnh xuất phát

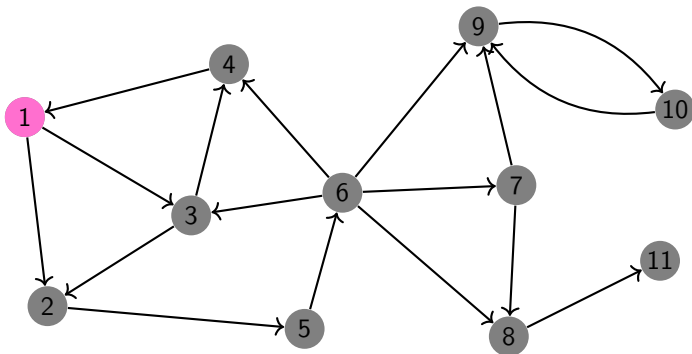
BFS: Ví dụ



Queue:

	1	2	3	4	5	6	7	8	9	10	11
Visited	F	F	F	F	F	F	F	F	F	F	F

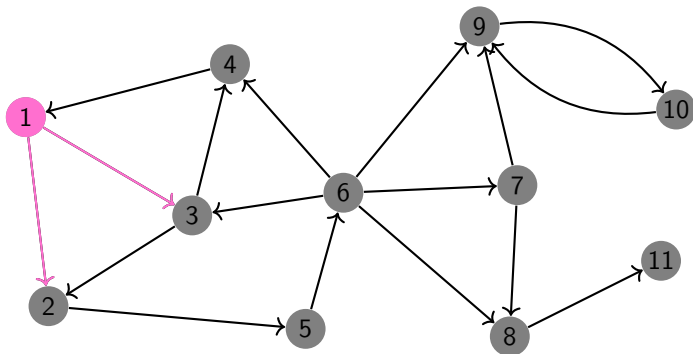
BFS: Ví dụ



Queue: 1

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	F	F	F	F	F	F	F	F	F	F

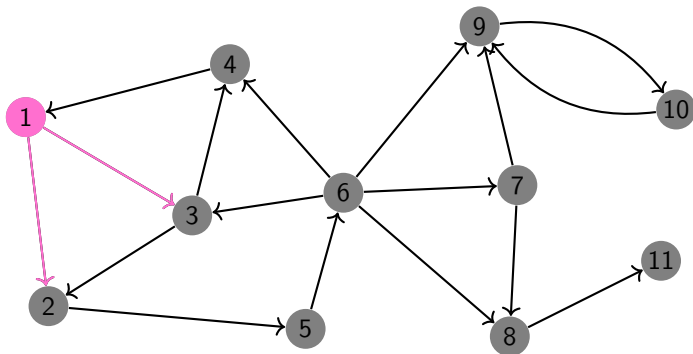
BFS: Ví dụ



Queue: 1

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	F	F	F	F	F	F	F	F	F	F

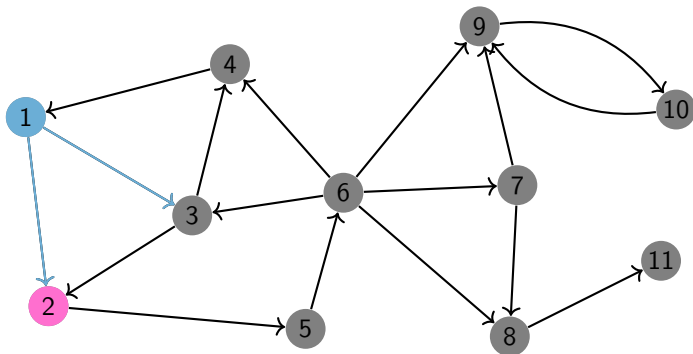
BFS: Ví dụ



Queue: 1 2 3

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	F	F	F	F	F	F	F	F

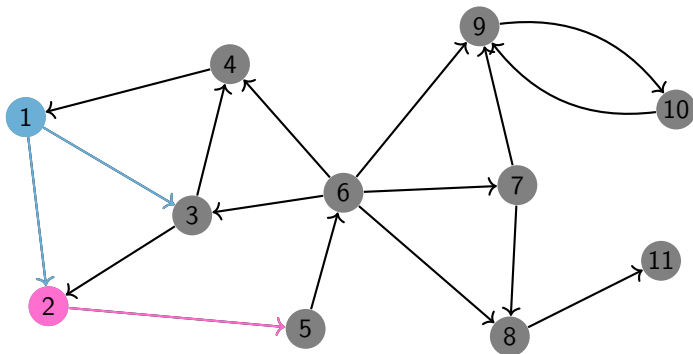
BFS: Ví dụ



Queue: 2 3

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	F	F	F	F	F	F	F	F

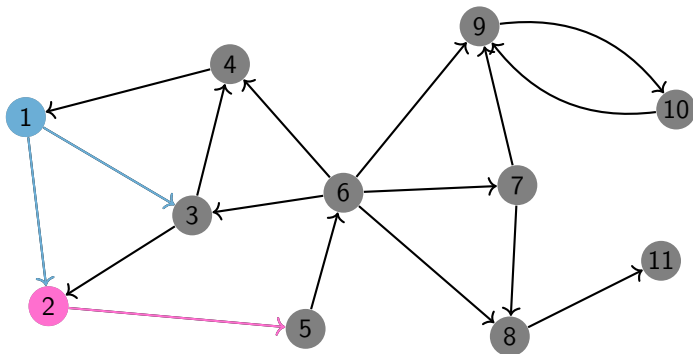
BFS: Ví dụ



Queue: 2 3

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	F	F	F	F	F	F	F	F

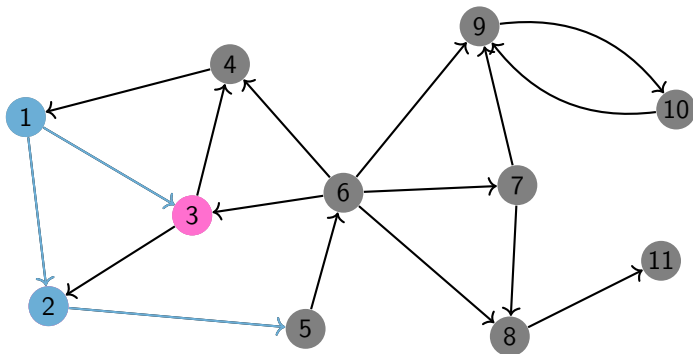
BFS: Ví dụ



Queue: 2 3 5

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	F	T	F	F	F	F	F	F

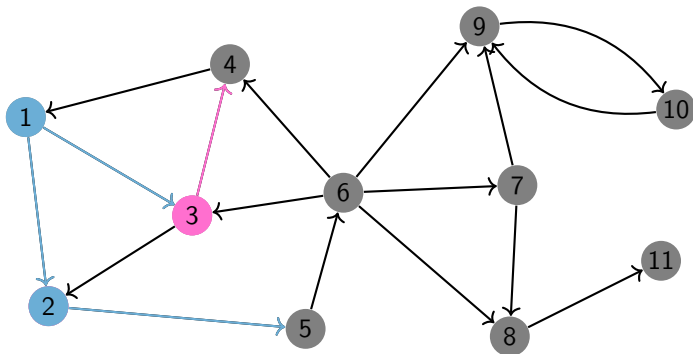
BFS: Ví dụ



Queue: 3 5

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	F	T	F	F	F	F	F	F

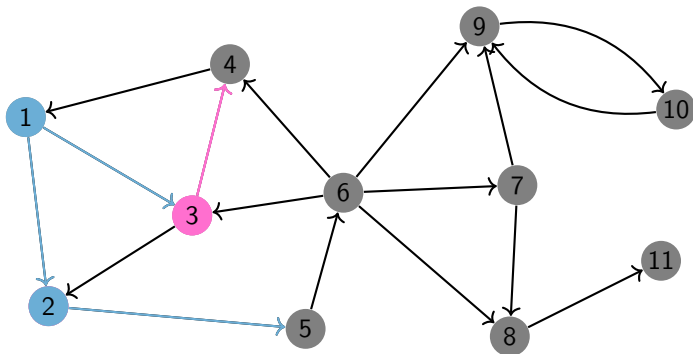
BFS: Ví dụ



Queue: 3 5

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	F	T	F	F	F	F	F	F

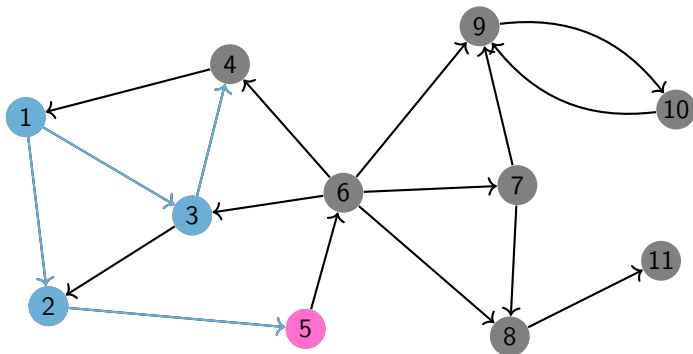
BFS: Ví dụ



Queue: 3 5 4

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	F	F	F	F	F	F

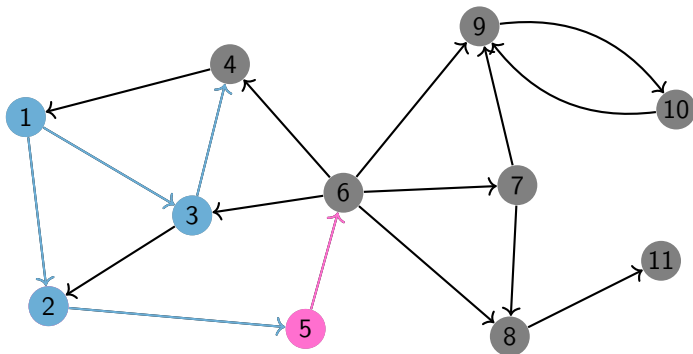
BFS: Ví dụ



Queue: 5 4

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	F	F	F	F	F	F

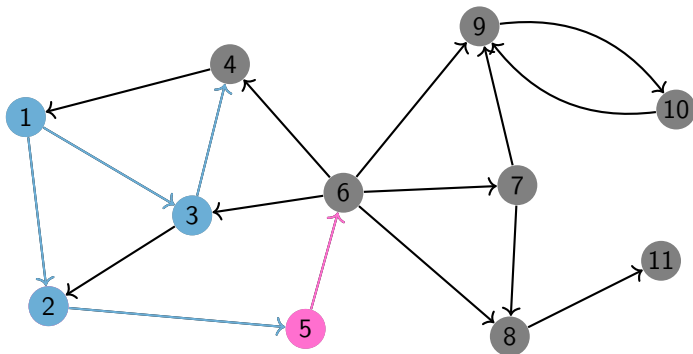
BFS: Ví dụ



Queue: 5 4

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	F	F	F	F	F	F

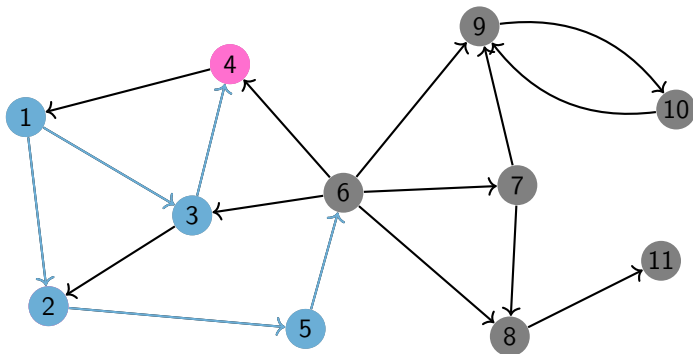
BFS: Ví dụ



Queue: 5 4 6

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	F	F	F	F	F

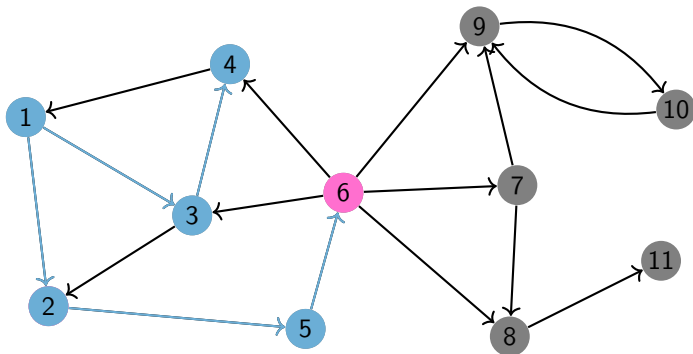
BFS: Ví dụ



Queue: 4 6

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	F	F	F	F	F

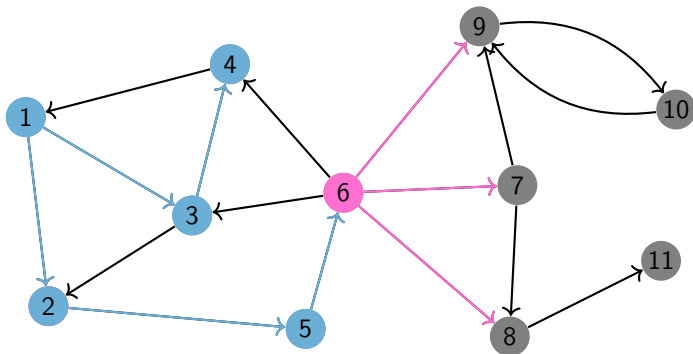
BFS: Ví dụ



Queue: 6

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	F	F	F	F	F

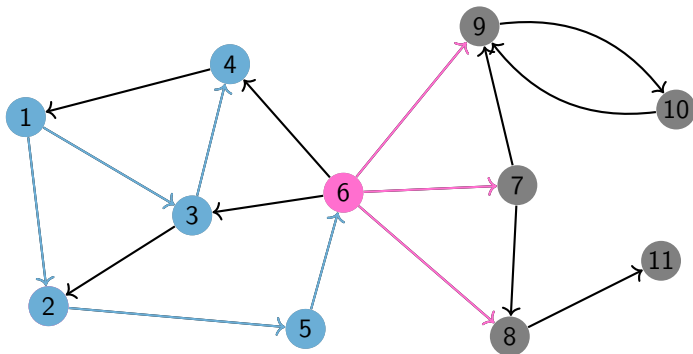
BFS: Ví dụ



Queue: 6

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	F	F	F	F	F

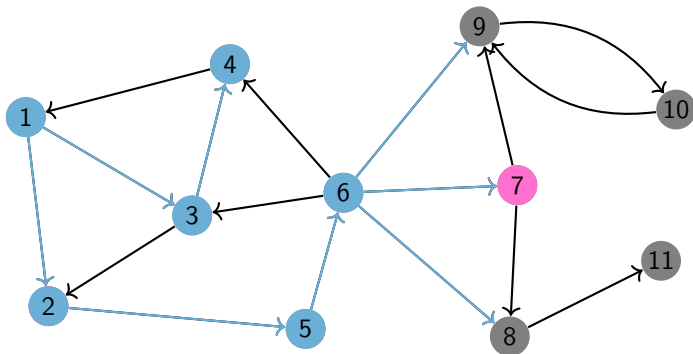
BFS: Ví dụ



Queue: 6 7 8 9

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	F	F

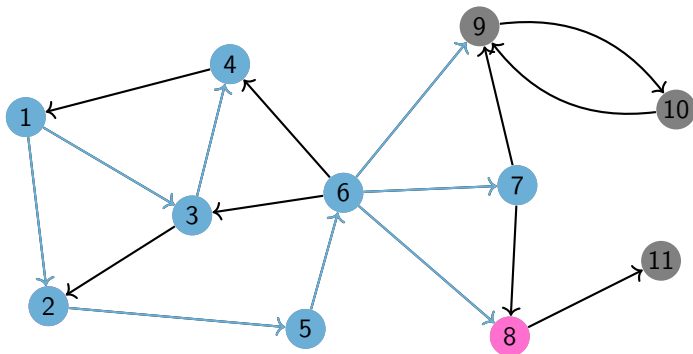
BFS: Ví dụ



Queue: 7 8 9

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	F	F

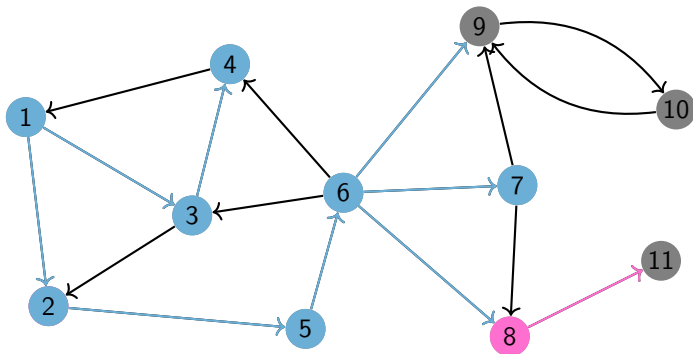
BFS: Ví dụ



Queue: 8 9

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	F	F

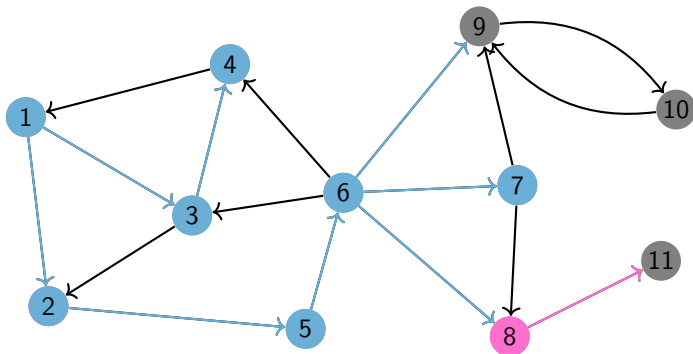
BFS: Ví dụ



Queue: 8 9

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	F	F

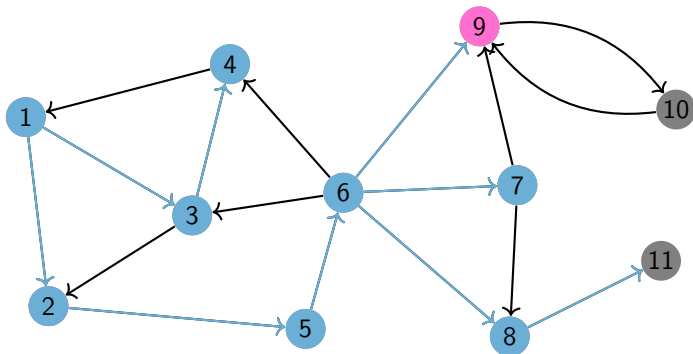
BFS: Ví dụ



Queue: 8 9 11

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	F	T

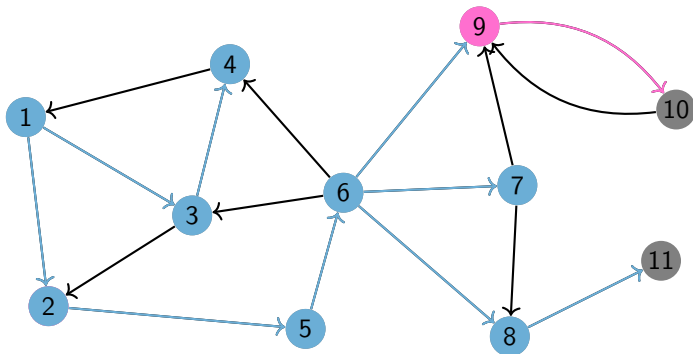
BFS: Ví dụ



Queue: 9 11

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	F	T

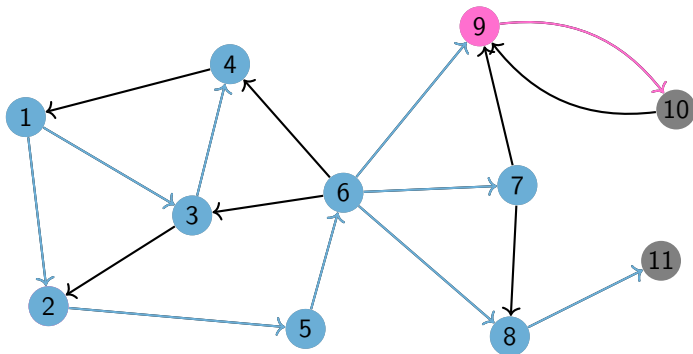
BFS: Ví dụ



Queue: 9 11

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	F	T

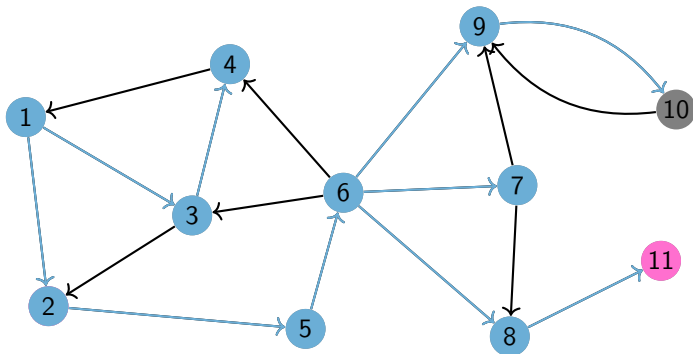
BFS: Ví dụ



Queue: 9 11 10

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	T

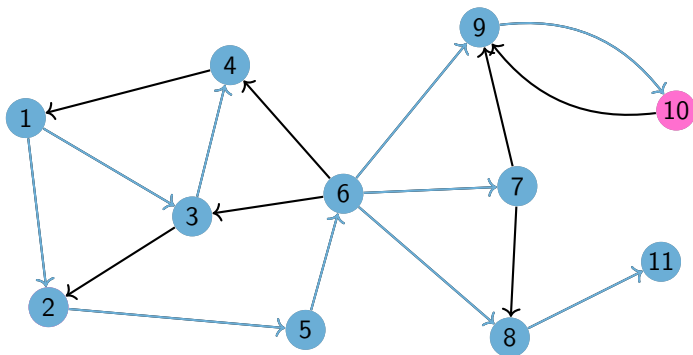
BFS: Ví dụ



Queue: 11 10

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	T

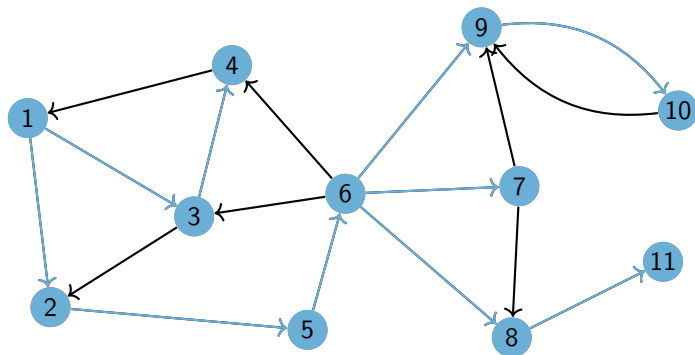
BFS: Ví dụ



Queue: 10

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	T

BFS: Ví dụ



Queue:

	1	2	3	4	5	6	7	8	9	10	11
Visited	T	T	T	T	T	T	T	T	T	T	T

BFS

```
1  vector<int> Adj[1001];
2  vector<bool> bVisited(1001, false);
3  queue<int> Q;
4  Q.push(start);
5  bVisited[start] = true;
6
7  while (!Q.empty()) {
8      int u = Q.front(); Q.pop();
9
10     for (int i = 0; i < Adj[u].size(); ++i) {
11         int v = Adj[u][i];
12         if (!bVisited[v]) {
13             Q.push(v);
14             bVisited[v] = true;
15         }
16     }
17 }
```

- 1 Cơ bản về đồ thị
- 2 Tìm kiếm theo chiều sâu và ứng dụng - DFS
- 3 Tìm kiếm theo chiều rộng và ứng dụng - BFS
 - Tìm kiếm theo chiều rộng - BFS
 - Đường đi ngắn nhất trên đồ thị không trọng số

Đường đi ngắn nhất trên đồ thị không trọng số

- Cho đồ thị không trọng số $G = (V, E)$, hãy tìm đường đi ngắn nhất từ đỉnh A đến đỉnh B
- Có nghĩa là tìm đường đi từ a đến b đi qua ít cạnh nhất
- BFS duyệt các đỉnh theo trình tự tăng dần khoảng cách từ đỉnh xuất phát
- Vì vậy chỉ cần gọi một lần BFS từ đỉnh a đến khi tìm thấy b
- Hoặc duyệt qua toàn bộ G , và ta có đường đi ngắn nhất từ a đến tất cả các đỉnh khác
- Độ phức tạp: $\mathcal{O}(n + m)$

Đường đi ngắn nhất trên đồ thị không trọng số

```
1  vector<int> Adj[1001];
2  vector<int> iDist(1001, -1);
3  queue<int> Q;
4  Q.push(a);
5  iDist[a] = 0;
6
7  while (!Q.empty()) {
8      int u = Q.front(); Q.pop();
9      for (int i = 0; i < Adj[u].size(); ++i) {
10         int v = Adj[u][i];
11         if (iDist[v] == -1) {
12             Q.push(v);
13             iDist[v] = 1 + iDist[u];
14         }
15     }
16 }
17 cout << iDist[b] << endl;
```